

Unit 4

PROCESSOR FUNDAMENTALS

92 items · 766 marks

Syllabus sections in this unit

- 4.1 Central Processing Unit (CPU) Architecture
- 4.2 Assembly Language
- 4.3 Bit manipulation

Unit 4: Processor Fundamentals

1. M/J 26/P11/Q3/a,b,c

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

instruction		explanation
opcode	operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n / Bn / &n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are:

address	data
100	200
101	205
102	100
103	50
...	
200	101
201	100
202	20
203	13
204	220
205	30

- (a) Complete the table to identify the data that will be stored in the accumulator after each set of instructions has been executed.

accumulator current data	set of instructions	new accumulator data
B11001010	ADD B00000010 INC ACC	
#100	LDM #200 ADD 200	
#103	LDD 103 DEC ACC DEC ACC	

[3]

(b) The instruction set also contains the following instructions:

instruction		explanation
opcode	operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
CSL	#n	Bits in ACC are shifted n places to the left using a cyclic shift
CSR	#n	Bits in ACC are shifted n places to the right using a cyclic shift
ASL	#n	Bits in ACC are shifted n places to the left using an arithmetic shift
ASR	#n	Bits in ACC are shifted n places to the right using an arithmetic shift
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(i) Write the bit manipulation instruction that will set the ACC to $\&0$

[2]

(ii) The ACC currently contains the following binary value.

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Write the result after the instruction `LSR #3` is run.

--	--	--	--	--	--	--	--

[1]

(c) Immediate, relative and direct are three modes of addressing used in assembly language programming.

Identify and describe **one** additional mode of addressing used in assembly language programming.

Include an example of an instruction that uses your selected mode of addressing in your answer.

mode of addressing

description with example

[3]

2. M/J 26/P13/Q7/a,b,c

7 A program is written in assembly language.

(a) Complete the table by writing the name of each mode of addressing.

mode of addressing	description
	The operand is an address where the data is stored.
	The operand is an address that stores the address where the data is stored.
	The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.
	The operand is the data to be used.

[4]

(b) Explain how bit manipulation can be used to monitor or control a device.

[3]

(c) Identify **one** similarity and **one** difference between a right logical shift and a right arithmetic shift.

similarity

difference

[2]

3. O/N 25/P11/Q4/a,b,c,d

4 The table shows part of the instruction set for a processor. The processor has one register, the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end.
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

(a) The ACC currently contains the following positive binary integer:

0	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---

Write a bit manipulation instruction that uses a binary shift to change the contents of the ACC to:

0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---

Instruction

[1]

(b) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Write the contents of the ACC after the instruction `XOR &12` is carried out.

--	--	--	--	--	--	--	--

[1]

(c) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Write the contents of the ACC after the instruction `AND #63` is carried out.

--	--	--	--	--	--	--	--

[1]

(d) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

The current contents of memory are:

Address	Data
98	00100100
99	00110001
100	00110011
101	10100011
102	10101100

Write the contents of the ACC after the instruction `OR 100` is carried out.

--	--	--	--	--	--	--	--

[1]

4. O/N 25/P11/Q6/a,b

- 6 (a) The processor uses several registers, including the Accumulator (ACC) and the Current Instruction Register (CIR).

Complete the table by describing the role of each register.

Register	Role
ACC	
CIR	

[2]

- (b) Increasing the number of cores in a processor can affect the performance of a computer.

Describe the drawbacks of increasing the number of cores in a processor.

[2]

5. O/N 25/P12/Q3/a,b,c

- 3 (a) (i) State what is meant by relative addressing.

[1]

- (ii) Registers such as the Accumulator (ACC) and the Index Register (IX) are used in the CPU.

Identify **two** special purpose registers used in the CPU. Do **not** include the ACC or IX in your answers.

1

2

[2]

- (b) The following table shows part of the instruction set for a processor. The processor has two registers: the ACC and an IX.

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127		

The current contents of the main memory and the index register are shown.

Address	Instruction
98	8
99	16
100	3
101	98
102	32
IX	2

Write the contents of the ACC after each instruction is executed.

Instruction	Value in ACC
LDM #98	
LDI 101	
LDX 100	

[3]

- (c) A student buys a new computer. The table shows the specifications of the old computer and the new computer.

Old computer	New computer
1.8 GHz dual core processor	2.3 GHz dual core processor
16 MB cache	32 MB cache

Explain why increasing the clock speed **and** increasing the cache memory will improve the performance of the computer.

Clock speed

Cache memory

[4]

6. O/N 25/P13/Q2/c

- (c) A three-place arithmetic shift to the right is performed on the following two's complement negative integer.

Show the result of this arithmetic shift.

10010011

[1]

7. O/N 25/P13/Q6/a,b,c

- 6 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to the ACC
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
CMP	#n	Compare the contents of the ACC with number n
CMP	<address>	Compare the contents of the ACC with the contents of <address>
LSL	#n	Bits in the ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in the ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

The current contents of main memory are shown:

Address	Data
100	0000 0011
101	1010 1110
102	1100 1100
103	1111 1111
104	1100 1100

(a) Complete the table by writing the contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction
0000 1111	AND 101	
0000 0000	LDM #100	
0000 0001	XOR &F1	
0001 0001	CMP 101	

[4]

(b) The Von Neumann model for a computer system uses registers.

Describe the role of the Memory Address Register (MAR) and Memory Data Register (MDR) in the fetch-execute (F-E) cycle.

[4]

(c) Assembly language instructions are grouped.

Complete each statement by writing the name of the appropriate instruction group.

Loading data into the accumulator is an example of an instruction in the

..... group. Incrementing the index register is an example of an instruction in the group. Branching to another address is an example of an instruction in the group.

[3]

8. M/J 25/P11/Q6/a(ii,iii)

6 A programmer is buying a new computer.

(a) The programmer is considering the following two computers:

Computer 1	Quad-core 2.2 GHz processor 16-bit architecture 1 GB Random Access Memory (RAM) 500 GB magnetic hard disk
Computer 2	Dual-core 3.8 GHz processor 32-bit architecture 2 GB RAM 500 GB solid state drive (SSD)

(ii) The two computers have different amounts of RAM.

Explain how different amounts of RAM affect the performance of a computer.

[3]

(iii) The two computers have different bus widths.

Explain how different bus widths affect the performance of a computer.

[2]

9. M/J 25/P11/Q8/a,b

8 (a) The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system
ACC denotes Accumulator <address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are:

address Instruction

80	10
81	8
82	80
83	81
...	
200	LDD 81
201	INC ACC
202	STO 83
203	LDI 82
204	CMP 83
205	JPE 209
206	LDD 83
207	ADD #10
208	JMP 210
209	DEC ACC
210	STO 81
211	END

Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81

[4]

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (i) Write the bit manipulation instruction that can be used to set the least significant bit to 1 in an 8-bit register. All other bits must remain unchanged.

The instruction needs to work on a register that contains any 8-bit binary number.

[1]

- (ii) The ACC currently contains the following binary value.

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Write the result after the instruction `XOR &FE` is run.

--	--	--	--	--	--	--	--

[1]

- (iii) The ACC currently contains the following binary value.

0	1	1	0	1	0	1	1
---	---	---	---	---	---	---	---

Write the result after the instruction `LSR #5` is run.

--	--	--	--	--	--	--	--

[1]

10. M/J 25/P12/Q1/a,b,c

- 1 (a) The table has six statements about the Von Neumann model for a computer system. Three of the statements are incorrect.

Statement number	Statement
1	The Program Counter (PC) stores the next instruction to be fetched from memory.
2	The Arithmetic and Logic Unit (ALU) performs mathematical and logical operations.
3	The Control Unit (CU) sends signals to other components on the data bus.
4	The Memory Data Register (MDR) transfers data to the memory address stored in the Memory Address Register (MAR).
5	The MAR stores an address from memory.
6	The Accumulator (ACC) stores the result of calculations.

Complete the table by writing the **three** incorrect statement numbers and the corrected statements.

Incorrect statement number	Corrected statement

[3]

(b) Registers that are used in the Fetch-Execute (F-E) cycle include the PC, MAR, MDR and the ACC.

Identify **one** other register and describe its role in the Fetch-Execute (F-E) cycle.

Register

Role

[2]

(c) Explain how an interrupt from an input device will be detected and handled in the F-E cycle.

[4]

11. M/J 25/P13/Q2/a,b,c

2 A computer has a processor.

(a) The processor has a Control Unit (CU) and system clock.

(i) Explain how the CU and the system clock work together.

[2]

(ii) The processor runs the Fetch-Execute (F-E) cycle.

Write the stages of the F-E cycle using register transfer notation.

[4]

(b) The computer has cache memory.

Describe **one** benefit of the computer using cache memory.

[2]

(c) The computer connects to a monitor using a High Definition Multimedia Interface (HDMI) cable that connects into an HDMI port.

Explain how HDMI provides connection to peripheral devices.

[2]

12. M/J 25/P13/Q5/a,b

5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

(a) The current contents of memory are:

Address	Instruction
50	47
51	48
52	49
53	50
...	
500	LDM #50
501	DEC ACC
502	CMP 51
503	JPE 505
504	JMP 501
505	STO 50
506	SUB #10
507	STO 51
508	END

Mode of addressing	Description
.....	
.....	

[4]

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Write the bit manipulation instructions that can be used to set only the most significant bit to 1 in an 8-bit register.

The instructions need to work on a register that contains any 8-bit binary number.

[2]

13. M/J 25/P13/Q7/a

7 (a) A computer can perform logical and arithmetic shifts.

(i) Show the result of a logical left shift of 2 places on the two's complement binary integer 11001010

[1]

(ii) Show the result of an arithmetic right shift of 3 places on the two's complement binary integer 10011110

[1]

14. O/N 24/P11/Q3/a,b

3 A computer system has a dual-core Central Processing Unit (CPU).

(a) State the purpose of the system clock and the Control Unit (CU) in a CPU.

System clock

CU

[2]

(b) (i) The number of cores in the processor affects the performance of the computer system.

Identify **one other** feature of a processor that can affect the performance of a computer system **and** state why it affects the performance.

Feature

Reason

[2]

(ii) A solid state (flash) memory drive is automatically recognised by the computer when it is plugged into a port in the computer.

Identify an appropriate type of port to connect the solid state memory drive to the computer.

Explain how this port provides an automatic connection.

Port

Explanation

[3]

15. O/N 24/P11/Q8/a,b

- 8** A computer designed using the Von Neumann model for a computer system contains general purpose registers and special purpose registers.

(a) Describe the purpose of the Status Register (SR).

[2]

(b) Identify **two** differences between general purpose registers and special purpose registers.

1

2

[2]

16. O/N 24/P12/Q3/a,b

3 A student has a computer.

(a) The computer is designed using the Von Neumann model for a computer system.

Complete the table by describing the purpose of each of the given registers.

Register	Purpose
Program Counter (PC)	
Memory Address Register (MAR)	
Memory Data Register (MDR)	
Index Register (IX)	

[4]

(b) The student needs to connect the computer to a monitor that has a screen resolution of 2560×1600 pixels. The monitor also has built-in speakers.

The computer has a Video Graphics Array (VGA) port and a High Definition Multimedia Interface (HDMI) port.

Explain the benefits of connecting the monitor to the computer using the HDMI port instead of the VGA port.

[4]

17. O/N 24/P12/Q8/a,b

8 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	25
20	23
21	2
22	4
23	15
24	50
25	22

The current contents of the ACC and IX are shown:

ACC	50
IX	20

Complete the table by writing the content of the ACC and the IX after each set of instructions has run.

	Instructions	ACC content	IX content
1	LDM #19 DEC ACC		
2	LDD 23 ADD 19		
3	LDI 25 INC ACC		
4	LDR #21 LDX 2		

[5]

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current content of the ACC is shown:

ACC	1	0	0	1	1	0	1	0
------------	---	---	---	---	---	---	---	---

(i) The table has three sets of instructions. The binary number 10011010 is reloaded into the ACC before each set of instructions is run.

Complete the table by writing the content of the ACC after each set of instructions has run.

	Instructions	ACC content
1	LSL #2	
2	ADD #5 AND #30	
3	OR B11110010 INC ACC	

[3]

- (ii) Explain how bit manipulation can be used to test whether the binary number stored in the ACC represents an odd denary number.

Write the bit manipulation instruction that will be used.

Explanation

Instruction

[3]

18. O/N 24/P13/Q7/a,b

- 7 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
50	54
51	55
52	50
53	52
54	100
55	25
56	50

The current contents of the ACC and IX are shown:

ACC	50
IX	45

Complete the table by writing the content of the ACC after each program has run.

	Instructions	ACC content
1	LDD 50 ADD #4 ADD 54	
2	LDI 53 DEC ACC ADD 56	
3	LDM #55 SUB #5	

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Explain how bit manipulation can be used to clear the data in an 8-bit register.

Write the bit manipulation instruction that will be used.

Explanation

Instruction

[3]

19. O/N 24/P13/Q8/c

(c) Describe the difference between a right logical binary shift and a right arithmetic binary shift.

[2]

20. M/J 24/P11/Q4/a,b

- 4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	24
20	2
21	1
22	3
23	5
24	4
25	22

The current contents of the ACC and IX are shown:

ACC	12
IX	1

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDD 20 ADD #2	
2	LDX 22	
3	LDI 25 INC ACC SUB 22	
4	LDD 19 LDM #5 LDM #25	

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
30	01110101
31	11111111
32	00000000
33	11001100
34	10101010

The current content of the ACC is shown:

1	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each program has run.

The binary number 10011010 is reloaded into the ACC before each program is run.

Program number	Code	ACC content
1	AND 31	
2	XOR B01001111	
3	OR #30	

21. M/J 24/P12/Q5/a,b

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (a) The current contents of memory are shown:

Address	Data
10	1
11	3
12	5
13	11
14	10
15	16
16	12

The current contents of the ACC and IX are shown:

ACC	10
IX	0

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDI 15 SUB #1	
2	LDD 14 ADD 11	
3	LDM #11 ADD #3 SUB 16	
4	LDR #2 LDX 14 ADD #2	

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
25	11000110
26	11100001
27	10000001
28	11001101
29	00001111

The current content of the ACC is shown:

0	1	0	0	0	1	1	0
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each program has run.

The binary number 01000110 is reloaded into the ACC before each program is run.

Program number	Code	ACC content
1	XOR 29	
2	AND #29	
3	OR B11111111	

22. M/J 24/P13/Q3/a,b

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		



- (a) The current contents of memory are shown:

Address	Data
48	51
49	6
50	48
51	50
52	49
53	50
54	6

The current contents of the ACC and IX are shown:

ACC	2
IX	50

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDM #50 INC ACC SUB #1	
2	LDI 51 ADD 52	
3	LDR #2 LDX 50 DEC ACC	
4	LDD 52 SUB 54 INC ACC	

[4]



(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
100	00001101
101	10111110
102	11110011
103	00110111
104	00000000

The current content of the ACC is shown:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---



Complete the table by writing the content of the ACC after each instruction has run.

The binary number 11111111 is reloaded into the ACC before each instruction is run.

Instruction number	Instruction	ACC content
1	LSL #2	
2	XOR 100	
3	AND 103	

[3]



23. O/N 23/P11/Q5/a,b,c

- 5 The Central Processing Unit (CPU) of the basic Von Neumann model for a computer system contains several special purpose registers.

- (a) The Memory Data Register (MDR), Index Register (IX) and the Accumulator (ACC) are examples of special purpose registers.

Identify **two other** special purpose registers **and** state their role in the CPU.

Special purpose register 1

Role

Special purpose register 2

Role

[4]

- (b) Describe what is meant by the **Immediate Access Store (IAS)** in a computer system.

[2]

- (c) A computer has a single 2.1 GHz CPU.

- (i) Describe how increasing the clock speed to 4 GHz can increase the performance of the computer.

[1]

- (ii) A second computer has a CPU with two 2.1 GHz cores.

Explain why the second computer does not always run twice as fast as the computer with one 2.1 GHz CPU.

[5]

24. O/N 23/P11/Q8/a,b,c

- 8 (a) Identify the purpose of the first pass of a two-pass assembler.

[1]

- (b) The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
OUT		Output to the screen the character whose ASCII value is stored in ACC
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123		

- (i) Give **one** example of an instruction that belongs to **each** of the following instruction groups.

Only use the instructions given in the table. Each instruction must have a suitable operand.

Data movement

Arithmetic operation

Conditional instruction

[3]

- (ii) The instruction `LDR #2` uses immediate addressing.

Give **one** similarity and **one** difference between direct addressing and indexed addressing.

Similarity

Difference

[2]

- (iii) Identify **one other** mode of addressing.

[1]

(c) The following table shows another part of the instruction set for the same processor.

Instruction		Explanation
Opcode	Operand	
AND	Bn	Bitwise AND operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the operand
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
# denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

0	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND B10100101

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSR #3

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

1	1	1	1	0	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

XOR B00100101

--	--	--	--	--	--	--	--

[1]

25. O/N 23/P12/Q8/c

8 A computer has an Operating System (OS).

(c) An optical disc reader/writer is connected to the computer.

(i) Give the name of **one** port that can provide a connection for the optical disc reader/writer.

[1]

(ii) Describe the roles of the address bus, the data bus **and** buffers in the process of writing data to the optical disc reader/writer.

[3]

26. O/N 23/P12/Q9/a,b

9 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	#n	Add the denary number n to the ACC
JMP	<address>	Jump to the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123		

(a) The instructions in the processor's instruction set can be grouped according to their function.

Identify the instruction group for each of the following opcodes.

IN

ADD

JPE

CMI

[4]

- (b)** The current contents of main memory and selected values from the ASCII character set are given on page 15.

Trace the program currently in memory using the trace table when the input is '1'.

Address	Instruction	Instruction address	ACC	IX	Memory address				Output
					100	101	110	111	
10	LDR #0				0	0	51	65	
11	IN								
12	STO 101								
13	LDX 110								
14	CMP 100								
15	JPE 21								
16	LDD 101								
17	ADD #16								
18	INC IX								
19	STO 100								
20	JMP 13								
21	OUT								
22	END								
...	↗								
100	0								
101	0								
...	↗								
110	51								
111	65								

ASCII value	Character
49	1
50	2
51	3
52	4
...	↗
65	A
66	B
67	C
68	D

[4]

27. O/N 23/P13/Q7/d

- 7** A laptop computer has Static RAM (SRAM).

A virtual reality headset and a laser printer are connected to the laptop.

- (d)** Identify **one** port that could be used to connect the virtual reality headset to the laptop.

Justify your choice.

Port

Justification

[3]

28. O/N 23/P13/Q9/a,b,c,d

- 9** A computer system is designed using the basic Von Neumann model.

Registers and buses are components in the Von Neumann model.

- (a) (i)** Identify **three other** components in the Von Neumann model of a computer system.

Do not include registers or buses in your answers.

1

2

3

[3]

- (ii)** Identify **two** differences between special purpose registers and general purpose registers.

1

2

[2]

- (b)** The following incomplete table contains steps of the Fetch-Execute (F-E) cycle and their descriptions.

Complete the table by writing the missing steps using register transfer notation **and** the missing descriptions.

Step	Description
.....	The address in PC is incremented.
$\text{MDR} \leftarrow [[\text{MAR}]]$	
$\text{MAR} \leftarrow [\text{PC}]$	
.....	The contents of MDR are copied into CIR.

[4]

- (c) Interrupts can be caused by software programs or hardware devices.

State **one** cause of a software interrupt.

[1]

- (d) The following statements describe the stages that the CPU performs when an interrupt is detected.

There are **three** missing statements.

Write the letter of the missing statements from the table in the correct place to complete the description.

- 1 At the end of each Fetch-Execute (F-E) cycle, the processor checks if an interrupt flag is set.
- 2
- 3 If the interrupt priority is high enough, the processor saves the current contents of the registers.
- 4
- 5 When servicing of the interrupt is complete, the processor restores the registers.

6

Letter	Stage
A	The address of the Interrupt Service (ISR) handling routine is loaded into the Program Counter (PC).
B	Lower priority interrupts are re-enabled.
C	The device causing the interrupt transfers data to the CPU.
D	The processor identifies the source of the interrupt and checks the priority of the interrupt.
E	The ISR is incremented.

[3]

29. M/J 23/P11/Q3/d(vi)

3 A computer has an Operating System (OS).

(d) The computer stores data in binary form.

(vi) Show the result of a 3-place right logical shift on the binary number:

11001100

[1]

30. M/J 23/P11/Q6

6 An interrupt is generated when a key is pressed on a computer keyboard.

Explain how the computer handles this interrupt.

[5]

31. M/J 23/P12/Q3/a,b

3 A program is written in assembly language.

(a) The program is converted into machine code by a two-pass assembler.

Draw **one or more** lines to identify the pass or passes in which each action takes place.

Action	Pass
generates object code	
reads the source code one line at a time	first
removes white space	second
adds labels to the symbol table	

[3]

(b) Assembly language statements can use different modes of addressing.

Complete the following description of addressing modes.

..... addressing is when the operand holds the memory address of the data.

..... addressing is when the operand holds a memory address that stores the memory address of the data.

..... addressing is when the operand is the data.

[3]

32. M/J 23/P12/Q4/d

4 Data in a computer is stored in binary form.

(d) Show the result of a 2-place left logical shift on the binary number:

01001111

[1]

33. M/J 23/P12/Q5/a,b,c

5 A student has purchased a new laptop.

(a) The laptop is designed using the Von Neumann model for a computer system.

Identify **two** types of signal that a control bus can transfer.

1

2

[2]

(b) Describe **two** ways the hardware of a laptop can be upgraded to improve the performance **and** explain how each upgrade improves the performance.

1

2

[4]

(c) Peripherals are connected to the laptop using ports.

(i) A printer is connected to a Universal Serial Bus (USB) port.

Describe how data is transmitted through a USB port.

[1]

(ii) A monitor is connected to the laptop using a different type of port.

Identify **one** other type of port that can be used to connect the monitor.

[1]

34. M/J 23/P13/Q7/b,c

7 A computer stores data in binary form.

(b) The computer has a Control Unit (CU), system clock and control bus.

Explain how the CU, system clock and control bus operate to transfer data between the components of the computer system.

[4]

(c) Complete the table by writing the register transfer notation for each stage of the Fetch-Execute (F-E) cycle given in the table.

Stage description	Register transfer notation
The Program Counter (PC) is incremented	
The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)	

[2]

35. O/N 22/P11/Q5/a,b,c,d

- 5 (a) State what is meant by the **stored program concept** in the Von Neumann model of a computer system.

[1]

- (b) A Central Processing Unit (CPU) contains several special purpose registers and other components.

- (i) State the role of the following registers.

Program Counter (PC)

Index Register (IX)

Status Register (SR)

[3]

- (ii) Tick (✓) **one** box in each row to identify the system bus used by each CPU component.

CPU component	Data bus	Address bus	Control bus
System clock			
Memory Address Register (MAR)			

[1]

- (iii) Describe the purpose of the Control Unit (CU) in a CPU.

[2]

- (c) Describe the purpose of an interrupt in a computer system.

[2]

- (d) Identify **two** causes of a software interrupt.

1

2

[2]

36. O/N 22/P11/Q6/a,b,c

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) The current contents of main memory and selected values from the ASCII character set are given.

(i) Trace the program currently in memory using the trace table.

Address	Instruction	Instruction address	ACC	IX	Memory address					Output
					100	101	110	111	112	
77	LDR #0				0	0	66	65	35	
78	LDX 110									
79	CMP #35									
80	JPE 92									
81	ADD 100									
82	STO 101									
83	LDM #1									
84	ADD 100									
85	STO 100									
86	INC IX									
87	LDX 110									
88	CMP #35									
89	JPN 81									
90	LDD 100									
91	ADD #48									
92	OUT									
93	END									
...										
100	0									
101	0									
...										
110	66									
111	65									
112	35									

ASCII value	Character
49	1
50	2
51	3
52	4
65	A
66	B
67	C
68	D

[4]

(ii) The following instructions are repeated for your reference.

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address

State the purpose of this part of an assembly language program.

```
LDD 100
STO 165
LDD 101
STO 100
LDD 165
STO 101
```

[1]

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

```
XOR B00011111
```

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

AND B11110000

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

OR B11001100

--	--	--	--	--	--	--	--

[1]

(iv) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

LSR #2

--	--	--	--	--	--	--	--

[1]

(c) Tick (✓) **one or more** boxes in each row to indicate whether the task is performed in the first pass or the second pass of a two-pass assembler.

Task	First pass	Second pass
Remove comments.		
Read the assembly language program one line at a time.		
Generate the object code.		
Check the opcode is in the instruction set.		

[2]

37. O/N 22/P11/Q7/d,e

- (d) Explain the reasons why increasing the amount of cache memory can improve the performance of a CPU.

[2]

- (e) State the name of a peripheral device port that provides a physical connection in the computer for each of these peripherals.

3D printer

Monitor

[2]

38. O/N 22/P12/Q7/a,b,c

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) Trace the program currently in memory using the trace table, stopping when line 90 is executed for a second time.

[5]

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The contents of memory addresses 50 and 51 are shown:

Memory address	Data value
50	01001101
51	10001111

(i) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

XOR B00011111

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND 50

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSL #3

--	--	--	--	--	--	--	--

[1]

(iv) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

OR 51

--	--	--	--	--	--	--	--

[1]

(c) Write the register transfer notation for each of the stages in the fetch-execute cycle described in the table.

Description	Register transfer notation
Copy the address of the next instruction into the Memory Address Register.	
Increment the Program Counter.	
Copy the contents of the Memory Data Register into the Current Instruction Register.	

[3]

39. O/N 22/P13/Q4/a,b,c

- 4 (a)** A Central Processing Unit (CPU) contains several special purpose registers and other components.

- (i)** State the roles of the following registers.

Memory Address Register (MAR)

Memory Data Register (MDR)

[2]

- (ii)** State when interrupts are detected during the Fetch-Execute (F-E) cycle.

[1]

- (b)** A computer system contains a system clock.

Describe the purpose of the system clock.

[2]

- (c)** Upgrading secondary storage to solid state typically improves the performance of computer systems.

Identify **one other** upgrade to the hardware **and** explain why it improves the performance of a computer system.

Upgrade

Explanation

[2]

40. O/N 22/P13/Q6/a,b,c

- 6** The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) The current contents of main memory and selected values from the ASCII character set are given.

(i) Trace the program currently in memory using the trace table.

Address	Instruction	Instruction address	ACC	IX	Memory address					Output
					100	101	110	111	112	
75	LDR #0				1	0	97	98	97	
76	LDD 100									
77	CMP #2									
78	JPE 91									
79	LDX 110									
80	SUB #32									
81	CMP #65									
82	JPN 86									
83	LDM #1									
84	ADD 101									
85	STO 101									
86	LDM #1									
87	ADD 100									
88	STO 100									
89	INC IX									
90	JMP 76									
91	LDD 101									
92	ADD #48									
93	OUT									
94	END									
...										
100	1									
101	0									
...										
110	97									
111	98									
112	97									

ASCII value	Character
49	1
50	2
51	3
52	4
65	A
66	B
67	C
68	D

[4]

(ii) Explain the purpose of **relative addressing** in an assembly language program.

[2]

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

0	0	1	1	0	1	1	0
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND B01001100

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

1	0	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

OR B01001111

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

1	0	0	1	1	1	0	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSR #2

--	--	--	--	--	--	--	--

[1]

(c) One instruction group is data movement.

Give the name of **one other** instruction group.

[1]

41. O/N 21/P11/Q6/a,b

6 (a) There are **two** errors in the following register transfer notation for the fetch-execute cycle.

- 1 MAR $\leftarrow$ [PC]
- 2 PC $\leftarrow$ [PC] - 1
- 3 MDR $\leftarrow$ [MAR]
- 4 CIR $\leftarrow$ [MDR]

Complete the following table by:

- identifying the line number of each error
- describing the error
- writing the correct statement.

Line number	Description of the error	Correct statement

[4]

(b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The current contents of main memory are shown:

Address	Data
100	00001111
101	11110000
102	01010101
103	11111111
104	00000000

Each row of the following table shows the current contents of ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
11111111	OR 101	
00000000	XOR #15	
10101010	LSR #2	
01010101	AND 104	

[4]

42. O/N 21/P12/Q7/a

7 A computer has system software.

(a) The Operating System handles interrupts.

Tick (✓) **one** box in each row to identify whether each event is an example of a hardware interrupt or a software interrupt.

Event	Hardware interrupt	Software interrupt
Buffer full		
Printer is out of paper		
User has pressed a key on the keyboard		
Division by zero		
Power failure		
Stack overflow		

[3]

43. O/N 21/P12/Q8/a,b

8 The Von Neumann model for a computer system uses registers.

(a) Describe the role of the following special purpose registers in the fetch-execute (F-E) cycle.

(i) Memory Address Register (MAR)

Memory Data Register (MDR)

[4]

(ii) Another special purpose register is the Index Register.

Identify **one other** special purpose register used in the Von Neumann model for a computer system.

[1]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123		

The current contents of main memory are shown:

Address	Data
100	01010101
101	11110000
102	00001111
103	00000000
104	11111111

- (i) In the following table, each row shows the current contents of the ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
01010101	XOR 101	
11110000	AND 104	
00001111	LSL #4	
11111111	OR 102	

[4]

- (ii) The following table contains five assembly language instruction groups.

Write an appropriate assembly language instruction for each instruction group, using the given instruction set. The first one has been completed for you.

Instruction Group	Instruction
Data movement	LDM #2
Input and output of data	
Arithmetic operations	
Unconditional and conditional instructions	
Compare instructions	

[4]

- (iii) The opcode **LDM** uses immediate addressing. The opcode **LDD** uses direct addressing.

Identify **and** describe **one additional** mode of addressing.

Mode of addressing

Description

[2]

44. O/N 21/P13/Q6/a,b

- 6 (a) There are **two** errors in the following register transfer notation for the fetch-execute cycle.

```
1  MAR ← [PC]

2  PC ← [PC] - 1

3  MDR ← [MAR]

4  CIR ← [MDR]
```

- Complete the following table by:
- identifying the line number of each error
 - describing the error
 - writing the correct statement.

Line number	Description of the error	Correct statement

[4]

(b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The current contents of main memory are shown:

Address	Data
100	00001111
101	11110000
102	01010101
103	11111111
104	00000000

Each row of the following table shows the current contents of ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
11111111	OR 101	
00000000	XOR #15	
10101010	LSR #2	
01010101	AND 104	

[4]

45. 9608 O/N 21/P11/Q6/a,b

- 6 (a) Complete the following sentences that describe parts of a processor in a Von Neumann model for a computer system.

There are buses that transfer data between components in a computer system.

The width of the determines the number of directly accessible memory locations.

The sends signals on the to direct the operation of system components.

..... pulses are used to synchronise the components on the motherboard.

[5]

- (b) Describe the stages of the fetch-execute (F-E) cycle.

[5]

46. 9608 O/N 21/P11/Q8

- 8 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to the ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to the ACC
STO	<address>	Store the contents of the ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address

The current contents of the main memory are:

Address	Instruction
130	LDI 160
131	ADD 153
132	STO 153
133	LDD 160
134	INC ACC
135	STO 160
136	JMP 130
...	
150	13
151	23
152	11
153	0
...	
160	150

The is incremented.

[5]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing: The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction
200	LDD 365
201	CMP 366
202	JPE 209
203	INC ACC
204	STO 365
205	MOV IX
206	LDX 365
207	OUT
208	JMP 200
209	END
...	
365	1
366	3
367	65
368	66
IX	0

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

Write the contents of the Accumulator after the processor has executed the following instruction:

LSL #2

--	--	--	--	--	--	--	--

[1]

(ii) The Accumulator currently contains the binary number:

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Identify the mathematical operation that the following instruction will perform on the contents of the accumulator.

LSR #3

[1]

48. M/J 21/P12/Q4/a,b(ii,iii)

- 4 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
# denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are:

Address	Instruction
70	IN
71	CMP 100
72	JPE 80
73	CMP 101
74	JPE 76
75	JMP 80
76	LDD 102
77	INC ACC
78	STO 102
79	JMP 70
80	LDD 102
81	DEC ACC
82	STO 102
83	JMP 70
...	
100	68
101	65
102	100

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

(a) Complete the trace table for the program currently in main memory when the following characters are input:

A D

(b) Some bit manipulation instructions are shown in the table:

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The contents of the memory address 300 are shown:

Bit Number	7	6	5	4	3	2	1	0
300	0	1	1	0	0	1	1	0

- (ii) An assembly language program needs to test if bit number 2 in memory address 300 is a 1.

Complete the assembly language instruction to perform this test.

#4

[1]

- (iii) An assembly language program needs to set bit numbers 4, 5, 6 and 7 to 0, but keep bits 0 to 3 with their existing values.

Write the assembly language instruction to perform this action.

[2]

49. M/J 21/P12/Q5/a,b

5 Seth uses a computer for work.

- (a) Complete the following descriptions of internal components of a computer by writing the missing terms.

The transmits the signals to coordinate events based on the electronic pulses of the

The carries data to the components, while the carries the address where data needs to be written to or read from.

The performs mathematical operations and logical comparisons.

[5]

- (b) Describe the ways in which the following factors can affect the performance of his laptop computer.

Number of cores

Clock speed

[4]

50. M/J 21/P13/Q3/a,b,c

- 3** A processor has one general purpose register, the Accumulator (ACC), and several special purpose registers.

- (a) Complete the following description of the role of the registers in the fetch-execute cycle by writing the missing registers.

The holds the address of the next instruction to be loaded. This address is sent to the

The holds the data fetched from this address.

This data is sent to the and the Control Unit decodes the instruction's opcode.

The is incremented.

[5]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing: The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction
200	LDD 365
201	CMP 366
202	JPE 209
203	INC ACC
204	STO 365
205	MOV IX
206	LDX 365
207	OUT
208	JMP 200
209	END
...	
365	1
366	3
367	65
368	66
IX	0

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

Write the contents of the Accumulator after the processor has executed the following instruction:

LSL #2

--	--	--	--	--	--	--	--

[1]

(ii) The Accumulator currently contains the binary number:

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Identify the mathematical operation that the following instruction will perform on the contents of the accumulator.

LSR #3

[1]

51. 9608 M/J 21/P11/Q3/a,b,d

- 3 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the content of the location at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare the contents of ACC with the contents of <address>.
JMP	<address>	Jump to the given address.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
STO	<address>	Store the contents of ACC at the given address.
END		Return control to the operating system.

The current contents of the main memory are:

Address	Instruction
100	LDD 200
101	CMP 201
102	JPE 106
103	DEC ACC
104	STO 200
105	JMP 101
106	END
...	
200	2
201	0
202	200

(a) Trace the program currently in memory using the following trace table.

[illegible]

[3]

(b) The instruction in memory address 100 needs to be changed. It needs to use indirect addressing to load the contents of memory address 200.

Give the new instruction to replace `LDD 200`.

[1]

(d) The status register contains condition flags.

Identify **three** condition flags that can be set in the status register.

- 1
- 2
- 3

[3]

52. 9608 M/J 21/P12/Q2

- 2 The following diagram shows four register notations and seven descriptions.

Draw **one** line from each register notation to its **most appropriate** description.

Register notation	Description
MDR	Holds the op code and operand of an instruction ready for it to be decoded
	Holds the address of the next instruction to be read
CIR	Holds flags that are set when the Arithmetic and Logic Unit (ALU) executes instructions
	Holds data read from, or to be written to, memory
MAR	Holds the current value in the Index Register
	Holds the address where data is to be written to or read from
PC	Holds the result of the last instruction executed by the ALU

[4]

53. 9608 M/J 21/P12/Q3/a,b,d

- 3 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
STO	<address>	Store the contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
JMP	<address>	Jump to the given address.
CMP	<address>	Compare the contents of ACC with the contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
END		Return control to the operating system.

The current contents of the main memory are:

Address Instruction

50	LDD 103
51	CMP 101
52	JPE 59
53	ADD 102
54	STO 102
55	LDD 100
56	INC ACC
57	STO 100
58	JMP 51
59	ADD 102
60	STO 102
61	END
...	
100	1
101	3
102	0
103	100

(a) Trace the program currently in memory using the following trace table.

[illegible]

[5]

(b) The instruction in memory address 50 needs to be changed to use direct addressing to load the contents of the memory location at address 100.

Give the new instruction to replace `LDI 103`.

[1]

(d) The assembly language program uses direct and indirect addressing.

Identify **two other** modes of addressing used in an assembly language program.

1

2

[2]

54. 9608 M/J 21/P13/Q3/a,b,d

- 3 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
JMP	<address>	Jump to the given address.
CMP	<address>	Compare the contents of ACC with the contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The current contents of the main memory and selected values from the ASCII character set are:

Address	Instruction
75	LDR #2
76	LDX 180
77	CMP #0
78	JPE 82
79	OUT
80	DEC IX
81	JMP 76
82	END
...	
180	0
181	41
182	71
183	40
184	70
185	43
186	69

ASCII code table (selected codes only)	
ASCII code	Character
36	\$
40	(
41	)
43	+
69	E
70	F
71	G

55. M/J 20/P11/Q6/a,b

6 A processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

(a) The table gives **three** assembly language instructions for loading data into the ACC. It also identifies the addressing mode used for each instruction.

	Instruction	Addressing mode
A	LDM #193	Immediate
B	LDD 193	Direct
C	LDX 193	Indexed

(i) State the contents of the Accumulator after each of the instructions **A**, **B** and **C** are run.

A

B

C

[3]

(ii) Name **two** other addressing modes.

1

2

[2]

(b) The ACC is a general purpose register. The IX is a special purpose register.

Identify **two** other special purpose registers used in the fetch-execute cycle **and** describe their role in the cycle.

Register 1

Role

Register 2

Role

[4]

56. M/J 20/P12/Q2/c(iii)

2 Amir has created a sound file using his desktop computer.

(c) Amir's computer has system software, including utility software and an operating system.

(iii) The peripheral devices are plugged into USB ports of the computer.

Describe **two** benefits of connecting the peripheral devices using a USB port.

1

2

[4]

57. M/J 20/P12/Q5/a,c

5 (a) The steps 1 to 6 describe the first pass of a two-pass assembler.

The following three statements are used to complete the sequence of steps.

A	If it is already in the symbol table, it checks to see if the absolute address is known
B	When it meets a symbolic address, it checks to see if it is already in the symbol table
C	If it is known, it is entered

Write one of the letters **A**, **B** or **C** in the appropriate step to complete the sequence.

1. The assembler reads the assembly language instructions

2.

3. If it is not, it adds it to the symbol table

4.

5.

6. If it is not known, it is marked as unknown.

[2]

(c) The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the denary number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare contents of the address given with the contents of ACC
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

Complete the trace table for the following assembly language program. The first instruction has been completed for you.

Address	Instruction
20	LDD 103
21	CMP 101
22	JPE 30
23	LDD 100
24	ADD 101
25	STO 100
26	LDD 103
27	INC ACC
28	STO 103
29	JMP 20
30	END
...	
100	1
101	2
102	3
103	0

[illegible]

58. M/J 20/P13/Q2/c

2 Billy has a laser printer.

(c) Billy's computer has several ports.

(i) State the purpose of a port.

[1]

(ii) Identify **one** type of port.

[1]

59. O/N 19/P11/Q1/a,b,c

1 Von Neumann is an example of a computer architecture.

(a) The diagram has registers used in Von Neumann architecture on the left and descriptions on the right.

Draw **one** line to match each register with its correct description.

Register	Description
Current Instruction Register	Stores the data that has just been read from memory, or is about to be written to memory
Memory Address Register	Stores the instruction that is being decoded and executed
Program Counter	Stores the address of the input device from which the processor accesses the instruction
Memory Data Register	Stores the address of the next instruction to be read
	Stores the address of the memory location about to be written to or read from

[4]

(b) Many components of the computer system transfer data between them using buses. One example of a bus is an address bus.

(i) Name **two** other buses that exist within a computer and give the purpose of each.

Bus 1

Purpose

Bus 2

Purpose

[4]

(ii) State the benefit of increasing the address bus width from 16 bits to 32 bits.

[1]

(c) The following statements describe features of a low-level language.

Complete the statements by writing the appropriate terms in the spaces.

A is a sequence of instructions that are given an identifier. These instructions may need to be executed several times.

A is an instruction that tells the assembler to do something. It is not a program instruction.

The processor's instruction set can be put into several groups. One of these groups is

[3]

60. O/N 19/P13/Q7

7 The following table has descriptions of modes of addressing.

Complete the table by writing the name of the addressing mode for each description.

Addressing mode	Description
	Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).
	Load the contents of the address held at the given address to ACC.
	Load the contents of the given address to ACC.
	Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.
	Load the given value directly to ACC.

[5]

61. M/J 19/P11/Q5/a,b,c

5 A simple program written in assembly language is translated using a two-pass assembler.

(a) The table contains some of the tasks performed by a two-pass assembler.

Tick (✓) **one** box in each row to indicate whether the task is performed at the first or second pass. The first row has been completed for you.

Task	First pass	Second pass
Creation of symbol table	✓	
Expansion of macros		
Generation of object code		
Removal of comments		

[2]

(b) The processor's instruction set can be grouped according to their function. For example, one group is modes of addressing.

Identify **two** other groups of instructions.

1

2

[2]

(c) The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC.
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the denary number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
CMP	#n	Compare contents of ACC with denary number n.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address	Instruction
20	LDM #0
21	STO 300
22	CMP #0
23	JPE 28
24	LDX 100
25	ADD 301
26	OUT
27	JMP 30
28	LDX 100
29	OUT
30	LDD 300
31	INC ACC
32	STO 300
33	INC IX
34	CMP #2
35	JPN 22
36	END
...	
100	65
101	67
102	69
103	69
104	68
...	
300	
301	33
IX	0

ASCII code table (Selected codes only)	
ASCII Code	Character
65	A
66	B
67	C
68	D
69	E
97	a
98	b
99	c
100	d
101	e

Trace the program currently in memory using the following trace table. The first instruction has been completed for you.

62. M/J 19/P12/Q3/a,b,c

3 The fetch-execute cycle is shown in register transfer notation.

```
01      MAR ← [PC]
02      PC ← [PC] - 1
03      MDR ← [MAR]
04      CIR ← [MAR]
```

(a) There are **three** errors in the fetch-execute cycle shown.

Identify the line number of each error and give the correction.

Line number

Correction

Line number

Correction

Line number

Correction

[3]

(b) A processor's instruction set can be grouped according to their function. For example, one group is the input and output of data.

Identify **two** other groups of instructions.

1

2

[2]

(c) The following table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the denary number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare contents of ACC with denary number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address Instruction

50	LDM #0
51	STO 401
52	LDX 300
53	CMP #0
54	JPE 62
55	ADD 400
56	OUT
57	LDD 401
58	INC ACC
59	STO 401
60	INC IX
61	JMP 52
62	END
...	
300	2
301	5
302	0
303	4
...	
400	64
401	
IX	0

ASCII code table (Selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D
69	E

Trace the program currently in memory using the following trace table.
The first instruction has been completed for you.

63. M/J 19/P13/Q4/a,b,c

4 A program is written in assembly language.

- (a) The op codes `LDM` and `LDD` are used to load a register. The op code `LDM` uses immediate addressing, and the op code `LDD` uses direct addressing.

Describe what happens when the following instructions are run.

`LDM #300`

`LDD 300`

[2]

- (b) Assembly language instructions can be grouped by their purpose.

The following table shows four assembly language instructions.

Tick (✓) **one** box in each row to indicate the group each instruction belongs to.

Instruction	Description	Jump instruction	Arithmetic operation	Data movement
<code>LDR #3</code>	Load the number 3 to the Index Register			
<code>ADD #2</code>	Add 2 to the Accumulator			
<code>JPN 22</code>	Move to the instruction at address 22			
<code>DEC ACC</code>	Subtract 1 from the Accumulator			

[3]

- (c) The processor handles interrupts within the fetch-execute cycle.

- (i) Give **one** example of a hardware interrupt and **one** example of a software interrupt.

Hardware

Software

[2]

- (ii) Explain how the processor handles an interrupt.

[5]

64. O/N 18/P11/Q4/a,b,d

- 4 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) (i) State what is meant by **direct addressing** and **indirect addressing**.

Direct addressing

Indirect addressing

[2]

- (ii) Explain how the instruction `ADD 20` can be interpreted as either direct or indirect addressing.

Direct addressing

Indirect addressing

[2]

- (b) The assembly language instructions in the following table use either symbolic addressing or absolute addressing.

Tick (✓) **one** box in each row to indicate whether the instruction uses symbolic or absolute addressing.

Instruction	Symbolic	Absolute
ADD 90		
CMP found		
STO 20		

[2]

- (d) The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are provided with a copy of the instruction set.

Address Instruction

70	LDX 200
71	OUT
72	STO 203
73	LDD 204
74	INC ACC
75	STO 204
76	INC IX
77	LDX 200
78	CMP 203
79	JPN 81
80	OUT
81	LDD 204
82	CMP 205
83	JPN 74
84	END
...	
200	130
201	133
202	130
203	0
204	0
205	2
IX	0

ASCII code table (selected codes only)

ASCII code	Character
127	?
128	!
129	“
130	*
131	\$
132	&
133	%
134	/

Instruction set

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

Complete the trace table for the given assembly language program.

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) (i) State what is meant by **absolute addressing** and **symbolic addressing**.

Absolute addressing

Symbolic addressing

[2]

- (ii) Give an example of an `ADD` instruction using both absolute addressing and symbolic addressing.

Absolute addressing

Symbolic addressing

[2]

- (b) (i) State what is meant by **indexed addressing** and **immediate addressing**.

Indexed addressing

Immediate addressing

[2]

- (ii) Give an example of an instruction that uses:

Indexed addressing

Immediate addressing

[2]

- (d) The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address Instruction

40	LDD 100
41	CMP 104
42	JPE 54
43	LDX 100
44	CMP 105
45	JPN 47
46	OUT
47	LDD 100
48	DEC ACC
49	STO 100
50	INC IX
51	JMP 41
52	
53	
54	END
...	
100	2
101	302
102	303
103	303
104	0
105	303

IX **ASCII code table (selected codes only)**

ASCII code	Character
300	/
301	*
302	-
303	+
304	^
305	=

This is a copy of the instruction set.

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

Complete the trace table for the given assembly language program.

Line 1 $\text{MDR} \leftarrow [\text{PC}]$

Line 2 $\text{PC} \leftarrow \text{PC} + 1$

Line 3 $\text{MDR} \leftarrow [\text{MAR}]$

Line 4 $\text{CIR} \leftarrow \text{PC}$

- (a) Identify the line numbers of **three** errors that the student has made. Write the correct notation for each error.

Line number of error	Correct notation

[3]

- (b) One stage of the FE cycle includes checking for interrupts.

- (i) Give **three** different events that can generate an interrupt.

1

2

3

[3]

- (ii) Explain how interrupts are handled during the fetch-execute cycle.

[5]

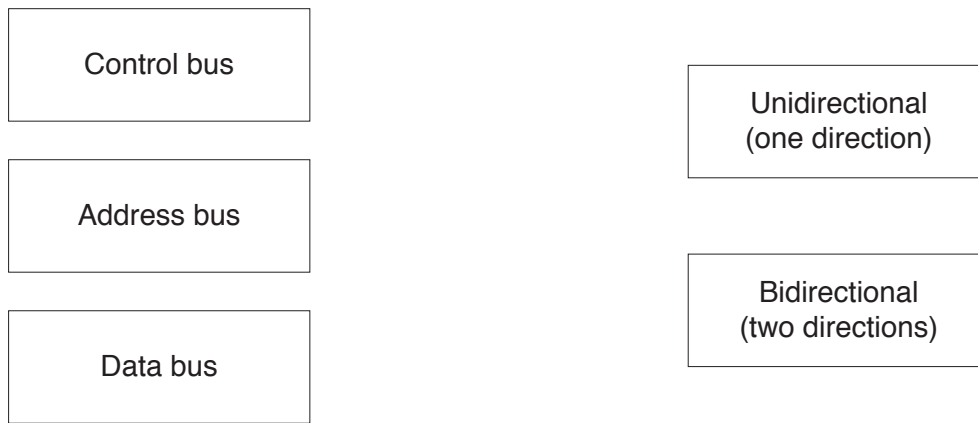
- (c) The processor uses buses in the FE cycle.

The diagram shows three buses and two descriptions.

Draw **one** line from each bus to its appropriate description.

Bus

Description



[2]

67. O/N 18/P13/Q2/a,b(iv),c

- 2 The following table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) State what is meant by **relative addressing** and **indexed addressing**.

Relative addressing

Indexed addressing

[2]

- (b) The current contents of a general purpose register (X) are:

X	1	1	1	1	0	0	1	0
---	---	---	---	---	---	---	---	---

(iv) Show the result on the general purpose register (X) after the following instruction is run.

INC X

--	--	--	--	--	--	--	--

[1]

(c) The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are provided with a copy of the instruction set.

Address Instruction

20	LDD 96
21	CMP 97
22	JPE 32
23	LDX 86
24	CMP 98
25	JPN 27
26	OUT
27	LDD 96
28	INC ACC
29	STO 96
30	INC IX
31	JMP 21
32	END
...	
93	453
94	453
95	452
96	8
97	10
98	453

IX

8

ASCII code table (selected codes only)

ASCII code	Character
450	<
451	>
452	=
453	&
454	(
455	)

Instruction set

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

Complete the trace table for the given assembly language program.

68. O/N 18/P13/Q6/a,b

6 The fetch-execute (FE) cycle uses special purpose registers.

(a) The stages in the FE cycle are shown in register transfer notation.

MAR $\leftarrow$ []

PC $\leftarrow$ PC + 1

..... $\leftarrow$ [[MAR]]

$\leftarrow$ [MDR]

(i) The steps shown in **part (a)** are incomplete.

Write the missing register names in the spaces in **part (a)**.

[3]

(ii) The third instruction [[MAR]] has double brackets.

State the purpose of the double brackets.

[1]

(b) One stage of the FE cycle includes checking for interrupts.

State what is meant by an **interrupt**.

[2]

69. M/J 18/P11/Q8/a,b

8 The Von Neumann model uses a series of registers.

(a) Explain what is meant by the term **register**.

[2]

(b) (i) Explain the purpose of the Memory Data Register (MDR).

[2]

(ii) Name **two** registers, other than the MDR, that are used in the fetch-execute cycle.

Register 1

Register 2

[2]

70. M/J 18/P13/Q3/a,b

3 A computer is designed using the Von Neumann model.

- (a)** Describe the role of the Arithmetic and Logic Unit (ALU) and Control Unit (CU) in the Von Neumann model.

ALU

CU

[4]

- (b)** Describe the role of the Status Register and Program Counter (PC).

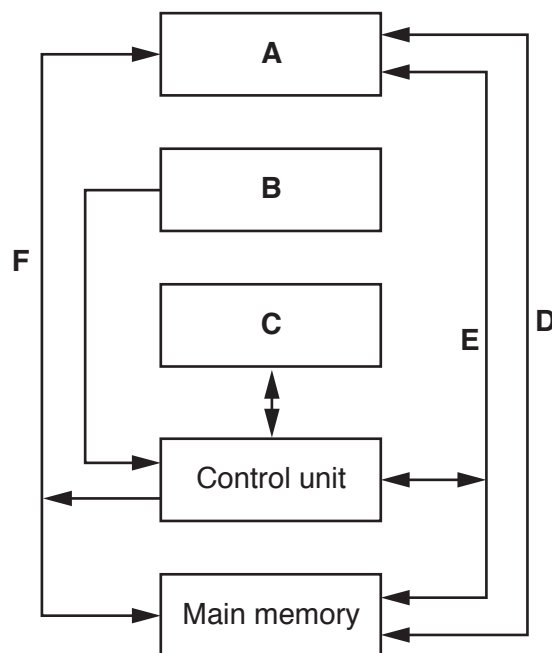
Status Register

PC

[4]

71. O/N 17/P12/Q4/a,b,d,e

4 The following diagram shows the components and buses found inside a typical personal computer (PC).



- (a)** Some components and buses only have labels **A** to **F** to identify them.

For each label, choose the appropriate title from the following list. The title for label **D** is already given.

- Control bus
- Address bus
- Arithmetic Logic Unit (ALU)
- General purpose registers
- Secondary storage
- System clock

A**B****C****D** Data bus**E****F**

[5]

(b) Clock speed is a factor that affects the performance of a PC. Explain this statement.

[2]

(d) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code (mnemonic)	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDV	#n	Relative addressing. Move to the address <i>n</i> locations from the address of the current instruction. Load the contents of this address to ACC.
STO	<address>	Store the contents of ACC at the given address.
INC		Increment the contents of ACC.
OUTCH		Output the character corresponding to the ASCII character code in ACC.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JMP	<address>	Jump to the given address.
CMP	#n	Compare the contents of ACC with number <i>n</i> .

Complete the trace table for the following assembly language program.

Label	Instruction
StartProg:	LDV #Offset
	CMP Value
	JPE EndProg
	OUTCH
	LDD Offset
	INC
	STO Offset
	JMP StartProg
EndProg:	END
Offset:	10
	50
	65
	89
	32
Value:	32

ASCII code table (selected codes only)				
<Space>	2	A	B	Y
32	50	65	66	89

Trace table:

ACC	Offset	OUTPUT
	10	
50		2
10		

[5]

- (e) The program given in **part (d)** is to be translated using a two-pass assembler. The program has been copied here for you.

Label	Instruction
StartProg:	LDV #Offset
	CMP Value
	JPE EndProg
	OUTCH
	LDD Offset
	INC
	STO Offset
	JMP StartProg
EndProg:	END
Offset:	10
	50
	65
	89
	32
Value:	32

On the first pass, the assembly process adds entries to a symbol table.

The following symbol table shows the first five entries, part way through the first pass.

The circular labels show the order in which the assembler made the entries to the symbol table.

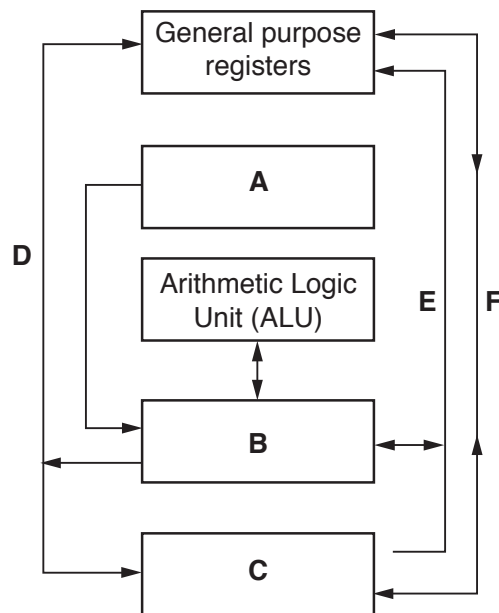
Complete the symbol table. Use circular labels to show the order in which the assembler makes the entries.

Symbol table

Symbolic address	Relative address
StartProg (1)	0 (2)
Offset (3)	UNKNOWN (4)
Value (5)	

72. O/N 17/P13/Q4/a,b,c,d

- 4 (a) The diagram shows the components and buses found inside a typical Personal Computer (PC).



Some components and buses only have labels **A** to **F** to identify them.

For each label, choose the appropriate title from the following list. The title for label **D** is already given.

- Control bus
- System clock
- Data bus
- Control unit
- Main memory
- Secondary storage

A

B

C

D Address bus

E

F

[5]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Op code (binary)	Explanation
Op code (mnemonic)	Operand		
LDM	#n	1100 0001	Immediate addressing. Load number <i>n</i> to ACC.
LDD	<address>	1100 0010	Direct addressing. Load the contents of the given address to ACC.
LDV	#n	1100 0011	Relative addressing. Move to the address <i>n</i> locations from the address of the current instruction. Load the contents of this address to ACC.
STO	<address>	1100 0100	Store the contents of ACC at the given address.
DEC		1100 0101	Decrement the contents of ACC.
OUTCH		1100 0111	Output the character corresponding to the ASCII character code in ACC.
JNE	<address>	1110 0110	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	1110 0011	(Unconditionally) jump to the given address.
CMP	#n	1110 0100	Compare the contents of ACC with number <i>n</i> .

Complete the trace table for the following assembly language program.

Label	Instruction
StartProg:	LDV #CountDown
	CMP Num1
	JNE CarryOn
	JMP Finish
CarryOn:	OUTCH
	LDD CountDown
	DEC
	STO CountDown
	JMP StartProg
Finish:	LDM #88
	OUTCH
	END
CountDown:	15
	32
	51
	67
Num1:	32

ASCII code table (selected codes only)				
<Space>	3	B	C	X
32	51	66	67	88

Trace table:

ACC	CountDown	OUTPUT
	15	
67		C
15		

[5]

(c) The program given in **part (b)** is to be translated using a two-pass assembler.

The program has been copied here for you. The program now starts with a directive which tells the assembler to load the first instruction of the program to address 100.

Label

	ORG	#0100
StartProg:	LDV	#CountDown
	CMP	Num1
	JNE	CarryOn
	JMP	Finish
CarryOn:	OUTCH	
	LDD	CountDown
	DEC	
	STO	CountDown
	JMP	StartProg
Finish:	LDM	#88
	OUTCH	
	END	
CountDown:		15
		32
		51
		67
Num1:		32

On the first pass of the two-pass process, the assembler adds entries to a symbol table.

The following symbol table shows the first eleven entries, part way through the first pass.

The circular labels show the order in which the assembler made the entries to the symbol table.

Symbol table

Symbolic address		Absolute address	
StartProg	1	100	2
CountDown	3	UNKNOWN	4
Num1	5	UNKNOWN	6
CarryOn	7	UNKNOWN	8 104 11
Finish	9	UNKNOWN	10

Explain how the assembler made these entries to the symbol table.

[3]

(d) The assembler software must then complete the second pass building up the executable file.

(i) Name the second table needed when the assembler software carries out the second pass.

[1]

The following shows two of the program instructions in machine code.

Instruction	Machine code	
	Binary	Hexadecimal
OUTCH	1100 0111	C7
JNE CarryOn	A	B

Each of the numbers **A** and **B** represents the complete instruction in two bytes, one byte for the op code and one byte for the operand.

(ii) Use the following instruction set to write the numbers for **A** and **B**.

A (binary)

B (hexadecimal)

[3]

Instruction		Op code (binary)	Explanation
Op code (mnemonic)	Operand		
LDM	#n	1100 0001	Immediate addressing. Load number <i>n</i> to ACC.
LDD	<address>	1100 0010	Direct addressing. Load the contents of the given address to ACC.
LDV	#n	1100 0011	Relative addressing. Move to the address <i>n</i> locations from the address of the current instruction. Load the contents of this address to ACC.
STO	<address>	1100 0100	Store the contents of ACC at the given address.
DEC		1100 0101	Decrement the contents of ACC.
OUTCH		1100 0111	Output the character corresponding to the ASCII character code in ACC.
JNE	<address>	1110 0110	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	1110 0011	(Unconditionally) jump to the given address.
CMP	#n	1110 0100	Compare the contents of ACC with number <i>n</i> .

73. M/J 17/P11/Q4/a,b,c,d(ii)

- 4 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Op code (binary)	Explanation
Op code (mnemonic)	Operand		
LDM	#n	0000 0001	Immediate addressing. Load the denary number <i>n</i> to ACC.
LDD	<address>	0000 0010	Direct addressing. Load the contents of the location at the given address to ACC.
LDI	<address>	0000 0101	Indirect addressing. At the given address is the address to be used. Load the contents of this second address to ACC.
LDX	<address>	0000 0110	Indexed addressing. Form the address from <address> + the contents of the Index Register (IX). Copy the contents of this calculated address to ACC.
LDR	#n	0000 0111	Immediate addressing. Load number <i>n</i> to IX.
STO	<address>	0000 1111	Store the contents of ACC at the given address.

The following diagram shows the contents of a section of main memory and the Index Register (IX).

- (a) Show the contents of the Accumulator (ACC) after each instruction is executed.

IX	0	0	0	0	0	0	1	1
----	---	---	---	---	---	---	---	---

		Address	Main Memory contents
(i) LDM #500	ACC	495	13
(ii) LDD 500	ACC	496	86
		497	92
(iii) LDX 500	ACC	498	486
		499	489
		500	496
(iv) LDI 500	ACC	501	497
		502	499
		503	502

- (b) Each machine code instruction is encoded as 16-bits (8-bit op code followed by an 8-bit operand).

Write the machine code for the following instructions:

LDM #17

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

LDX #97

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

[3]

- (c) Using an 8-bit operand, state the maximum number of memory locations, in denary, that can be directly addressed.

[1]

- (d) Computer scientists often write binary representations in hexadecimal.

- (ii) A second instruction has been written in hexadecimal as:

05 3F

Write the equivalent assembly language instruction, with the operand in denary.

[2]

74. M/J 17/P12/Q5/a,b,c(ii)

- 5 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Op code (binary)	Explanation
Op code (mnemonic)	Operand		
LDD <address>		0001 0011	Direct addressing. Load the contents of the location at the given address to the Accumulator (ACC).
LDI <address>		0001 0100	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
LDX <address>		0001 0101	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDM #n		0001 0010	Immediate addressing. Load the denary number n to ACC.
LDR #n		0001 0110	Immediate addressing. Load denary number n to the Index Register (IX).
STO <address>		0000 0111	Store the contents of ACC at the given address.

The following diagram shows the contents of a section of main memory and the Index Register (IX).

(a) Show the contents of the Accumulator (ACC) after each instruction is executed.

IX	0	0	0	0	0	1	1	0
----	---	---	---	---	---	---	---	---

(i) LDD 355

ACC [1]

(ii) LDM #355

ACC [1]

(iii) LDX 351

ACC [1]

(iv) LDI 355

ACC [1]

Address	Main memory contents
350	
351	86
352	
353	
354	
355	351
356	
357	22
358	

(b) Each machine code instruction is encoded as 16 bits (8-bit op code followed by an 8-bit operand).

Write the machine code for these instructions:

LDM #67

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

LDX #7

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

[3]

(c) Computer scientists often write binary representations in hexadecimal.

(ii) A second instruction has been written in hexadecimal as:

16 4D

Write the assembly language for this instruction with the operand in denary.

[2]

75. M/J 17/P13/Q4/a,b,c,d(ii)

- 4 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Op code (binary)	Explanation
Op code (mnemonic)	Operand		
LDM	#n	0000 0001	Immediate addressing. Load the denary number <i>n</i> to ACC.
LDD	<address>	0000 0010	Direct addressing. Load the contents of the location at the given address to ACC.
LDI	<address>	0000 0101	Indirect addressing. At the given address is the address to be used. Load the contents of this second address to ACC.
LDX	<address>	0000 0110	Indexed addressing. Form the address from <address> + the contents of the Index Register (IX). Copy the contents of this calculated address to ACC.
LDR	#n	0000 0111	Immediate addressing. Load number <i>n</i> to IX.
STO	<address>	0000 1111	Store the contents of ACC at the given address.

The following diagram shows the contents of a section of main memory and the Index Register (IX).

(a) Show the contents of the Accumulator (ACC) after each instruction is executed.

IX

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

		Address	Main Memory contents
(i) LDM #500			
ACC	[1]	495	13
(ii) LDD 500		496	86
ACC	[1]	497	92
(iii) LDX 500		498	486
ACC	[1]	499	489
		500	496
(iv) LDI 500		501	497
ACC	[1]	502	499
		503	502

- (b) Each machine code instruction is encoded as 16-bits (8-bit op code followed by an 8-bit operand).

Write the machine code for the following instructions:

LDM #17

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

LDX #97

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

[3]

- (c) Using an 8-bit operand, state the maximum number of memory locations, in denary, that can be directly addressed.

[1]

- (d) Computer scientists often write binary representations in hexadecimal.

- (ii) A second instruction has been written in hexadecimal as:

05 3F

Write the equivalent assembly language instruction, with the operand in denary.

[2]

76. O/N 16/P11/Q8/a,b

- 8 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
CMP	<address>	Compare contents of ACC with contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The diagram shows the contents of the main memory:

Main memory	
800	0110 0100
801	0111 1100
802	1001 0111
803	0111 0011
804	1001 0000
805	0011 1111
806	0000 1110
807	1110 1000
808	1000 1110
809	1100 0010
:	
:	
2000	1011 0101

(a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 802

Accumulator:

--	--	--	--	--	--	--	--

[1]

(ii) Show the contents of the Accumulator after execution of the instruction:

LDX 800

Index Register:

0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

[3]

- (b) (i) Complete the trace table below for the following assembly language program. This program contains denary values.

100	LDD 800
101	ADD 801
102	STO 802
103	LDD 803
104	CMP 802
105	JPE 107
106	JPN 110
107	STO 802
108	OUT
109	JMP 112
110	LDD 801
111	OUT
112	END
:	
:	
800	40
801	50
802	0
803	90

Selected values from the ASCII character set:

ASCII code	40	50	80	90	100
Character	(	2	P	Z	d

Trace table:

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	

[4]

(ii) There is a redundant instruction in the code in **part (b)(i)**.

State the address of this instruction.

[1]

77. O/N 16/P12/Q5/a,b

- 5 The table shows assembly language instructions for a processor that has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op Code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Index addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
STO	<address>	Store the contents of ACC at the given address.
INC	<register>	Add 1 to contents of the register (ACC or IX).
ADD	<address>	Add the contents of the given address to the ACC.
END		Return control to the operating system.

The diagram shows the contents of a section of main memory:

Main memory

100	0000 0010
101	1001 0011
102	0111 0011
103	0110 1011
104	0111 1110
105	1011 0001
106	0110 1000
107	0100 1011
...	
200	1001 1110

- (a) (i) Show the contents of the Accumulator after the execution of the instruction:

LDD 102

ACC:

--	--	--	--	--	--	--	--

[1]

- (ii) Show the contents of the Accumulator after the execution of the instruction:

LDX 101

IX:

0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---

ACC:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

[2]

- (iii) Show the contents of the Accumulator after the execution of the instruction:

LDI 103

ACC:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

[3]

(b) Complete the trace table below for the following assembly language program.

800	LDD 810
801	INC ACC
802	STO 812
803	LDD 811
804	ADD 812
805	STO 813
806	END
...	
810	28
811	41
812	0
813	0

Trace table:

ACC	Memory address			
	810	811	812	813
	28	41	0	0

[6]

78. O/N 16/P13/Q8/a,b

- 8 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
CMP	<address>	Compare contents of ACC with contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The diagram shows the contents of the main memory:

Main memory	
800	0110 0100
801	0111 1100
802	1001 0111
803	0111 0011
804	1001 0000
805	0011 1111
806	0000 1110
807	1110 1000
808	1000 1110
809	1100 0010
:	
:	
2000	1011 0101

(a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 802

Accumulator:

--	--	--	--	--	--	--	--

[1]

(ii) Show the contents of the Accumulator after execution of the instruction:

LDX 800

Index Register:

0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

[3]

- (b) (i) Complete the trace table below for the following assembly language program. This program contains denary values.

100	LDD 800
101	ADD 801
102	STO 802
103	LDD 803
104	CMP 802
105	JPE 107
106	JPN 110
107	STO 802
108	OUT
109	JMP 112
110	LDD 801
111	OUT
112	END
:	
:	
800	40
801	50
802	0
803	90

Selected values from the ASCII character set:

ASCII code	40	50	80	90	100
Character	(	2	P	Z	d

Trace table:

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	

[4]

(ii) There is a redundant instruction in the code in **part (b)(i)**.

State the address of this instruction.

[1]

79. M/J 16/P11/Q3/a,b

3 (a) Describe how special purpose registers are used in the fetch stage of the fetch-execute cycle.

[4]

(b) Use the statements A, B, C and D to complete the description of how the fetch-execute cycle handles an interrupt.

A	the address of the Interrupt Service Routine (ISR) is loaded to the Program Counter (PC).
B	the processor checks if there is an interrupt.
C	when the ISR completes, the processor restores the register contents.
D	the register contents are saved.

At the end of the cycle for the current instruction

If the interrupt flag is set,, and

The interrupted program continues its execution.

[4]

80. M/J 16/P11/Q9/a,b

9 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

(a) The diagram shows the current contents of a section of main memory and the index register:

60	0011 0010
61	0101 1101
62	0000 0100
63	1111 1001
64	0101 0101
65	1101 1111
66	0000 1101
67	0100 1101
68	0100 0101
69	0100 0011
...	
1000	0110 1001

Index register:

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

(i) Show the contents of the Accumulator after the execution of the instruction:

LDX 60

Accumulator:

--	--	--	--	--	--	--	--

Show how you obtained your answer.

[2]

(ii) Show the contents of the index register after the execution of the instruction:

DEC IX

Index register:

--	--	--	--	--	--	--	--

[1]

(b) Complete the trace table on the opposite page for the following assembly language program.

50	LDD 100
51	ADD 102
52	STO 103
53	LDX 100
54	ADD 100
55	CMP 101
56	JPE 58
57	JPN 59
58	OUT
59	INC IX
60	LDX 98
61	ADD 101
62	OUT
63	END
...	
100	20
101	100
102	1
103	0

IX (Index Register)

1

Selected values from the ASCII character set:

ASCII Code	118	119	120	121	122	123	124	125
Character	v	w	x	y	z	{		}

Trace table:

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50								
51								
52								
53								
54								
55								

[7]

81. M/J 16/P12/Q3/a,b

- 3 (a) Describe how special purpose registers are used in the fetch stage of the fetch-execute cycle.

[4]

- (b) Use the statements A, B, C and D to complete the description of how the fetch-execute cycle handles an interrupt.

A	the address of the Interrupt Service Routine (ISR) is loaded to the Program Counter (PC).
B	the processor checks if there is an interrupt.
C	when the ISR completes, the processor restores the register contents.
D	the register contents are saved.

At the end of the cycle for the current instruction

If the interrupt flag is set,, and

The interrupted program continues its execution.

[4]

82. M/J 16/P12/Q9/a,b

- 9 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) The diagram shows the current contents of a section of main memory and the index register:

60	0011 0010
61	0101 1101
62	0000 0100
63	1111 1001
64	0101 0101
65	1101 1111
66	0000 1101
67	0100 1101
68	0100 0101
69	0100 0011
...	
1000	0110 1001

Index register:

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

- (i) Show the contents of the Accumulator after the execution of the instruction:

LDX 60

Accumulator:

--	--	--	--	--	--	--	--

Show how you obtained your answer.

[2]

(ii) Show the contents of the index register after the execution of the instruction:

DEC IX

Index register:

--	--	--	--	--	--	--	--

[1]

(b) Complete the trace table on the opposite page for the following assembly language program.

50	LDD 100
51	ADD 102
52	STO 103
53	LDX 100
54	ADD 100
55	CMP 101
56	JPE 58
57	JPN 59
58	OUT
59	INC IX
60	LDX 98
61	ADD 101
62	OUT
63	END
...	
100	20
101	100
102	1
103	0

IX (Index Register)

1

Selected values from the ASCII character set:

ASCII Code	118	119	120	121	122	123	124	125
Character	v	w	x	y	z	{		}

Trace table:

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50								
51								
52								
53								
54								
55								

[7]

83. M/J 16/P13/Q4/a,b

- 4 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The diagram shows the contents of the index register:

Index register:	1	1	0	0	1	1	0	1
-----------------	---	---	---	---	---	---	---	---

(a) Show the contents of the index register after the execution of the instruction:

INC IX

Index register:								
-----------------	--	--	--	--	--	--	--	--

[1]

(b) Complete the trace table on the opposite page for the following assembly language program.

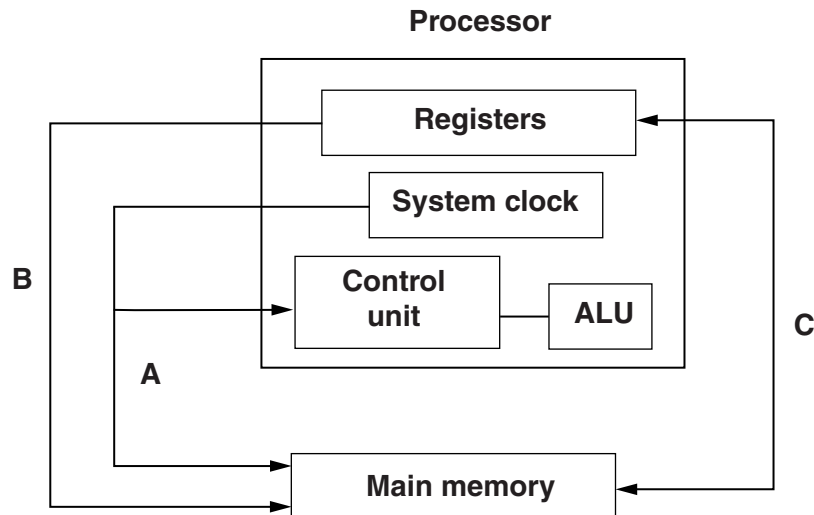
20	LDX 90
21	DEC ACC
22	STO 90
23	INC IX
24	LDX 90
25	DEC ACC
26	CMP 90
27	JPE 29
28	JPN 31
29	ADD 90
30	OUT
31	ADD 93
32	STO 93
33	OUT
34	END
:	
:	
90	2
91	90
92	55
93	34
IX	2

Selected values from the ASCII character set:

ASCII Code	65	66	67	68	69	70	71	72
Character	A	B	C	D	E	F	G	H

Trace table:

Instruction	Working space	ACC	Memory address				IX	OUTPUT
			90	91	92	93		
			2	90	55	34	2	
20								
21								
22								
23								
24								
25								
26								

84. O/N 15/P11/Q2/a,b**2 (a)**

The diagram above shows a simplified form of processor architecture.

Name the three buses labelled A, B and C.

A

B

C

[3]

(b) State the role of each of the following special purpose registers used in a typical processor.

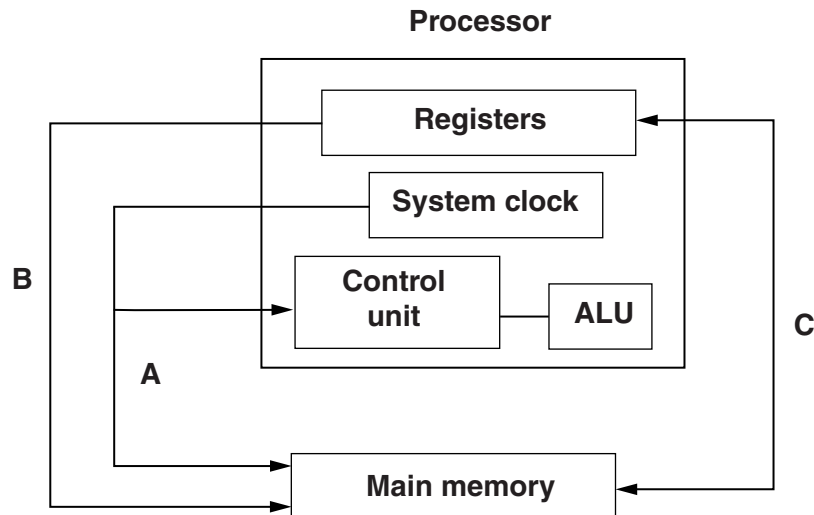
Program Counter

Memory Data Register

Current Instruction Register

Memory Address Register

[4]

85. O/N 15/P13/Q2/a,b**2 (a)**

The diagram above shows a simplified form of processor architecture.

Name the three buses labelled A, B and C.

A

B

C

[3]

(b) State the role of each of the following special purpose registers used in a typical processor.

Program Counter

Memory Data Register

Current Instruction Register

Memory Address Register

[4]

86. M/J 15/P11/Q3

3 Five modes of addressing and five descriptions are shown below.

Draw a line to connect **each** mode of addressing to its correct description.

Mode of addressing**Description**

direct

the operand is the address of the address of the value to be used

immediate

the operand is the address of the value to be used

indexed

the operand is the offset from the current address where the value to be used is stored

indirect

the operand plus the contents of the index register is the address of the value to be used

relative

the operand is the value to be used

[4]

87. M/J 15/P11/Q8/a,b,c

- 8 (a)** Explain how the width of the data bus and system clock speed affect the performance of a computer system.

Width of the data bus

Clock speed

[3]

- (b)** Most computers use Universal Serial Bus (USB) ports to allow the attachment of devices.

Describe **two** benefits of using USB ports.

1

2

[2]

- (c)** The table shows six stages in the von Neumann fetch-execute cycle.

Put the stages into the correct sequence by writing the numbers 1 to 6 in the right hand column.

Description of stage	Sequence number
the instruction is copied from the Memory Data Register (MDR) and placed in the Current Instruction Register (CIR)	
the instruction is executed	
the instruction is decoded	
the address contained in the Program Counter (PC) is copied to the Memory Address Register (MAR)	
the value in the Program Counter (PC) is incremented so that it points to the next instruction to be fetched	
the instruction is copied from the memory location contained in the Memory Address Register (MAR) and is placed in the Memory Data Register (MDR)	

[6]

88. M/J 15/P12/Q3

3 Five modes of addressing and five descriptions are shown below.

Draw a line to connect **each** mode of addressing to its correct description.

Mode of addressing**Description**

direct

the operand is the address of the address of the value to be used

immediate

the operand is the address of the value to be used

indexed

the operand is the offset from the current address where the value to be used is stored

indirect

the operand plus the contents of the index register is the address of the value to be used

relative

the operand is the value to be used

[4]

89. M/J 15/P12/Q8/a,b,c

- 8 (a)** Explain how the width of the data bus and system clock speed affect the performance of a computer system.

Width of the data bus

Clock speed

[3]

- (b)** Most computers use Universal Serial Bus (USB) ports to allow the attachment of devices.

Describe **two** benefits of using USB ports.

1

2

[2]

- (c)** The table shows six stages in the von Neumann fetch-execute cycle.

Put the stages into the correct sequence by writing the numbers 1 to 6 in the right hand column.

Description of stage	Sequence number
the instruction is copied from the Memory Data Register (MDR) and placed in the Current Instruction Register (CIR)	
the instruction is executed	
the instruction is decoded	
the address contained in the Program Counter (PC) is copied to the Memory Address Register (MAR)	
the value in the Program Counter (PC) is incremented so that it points to the next instruction to be fetched	
the instruction is copied from the memory location contained in the Memory Address Register (MAR) and is placed in the Memory Data Register (MDR)	

[6]

90. M/J 15/P13/Q2

- 2** Assemblers translate from assembly language to machine code. Some assemblers scan the assembly language program twice; these are referred to as two-pass assemblers.

The following table shows five activities performed by two-pass assemblers.

Write **1** or **2** to indicate whether the activity is carried out during the first pass or during the second pass.

Activity	First pass or second pass
any symbolic address is replaced by an absolute address	
any directives are acted upon	
any symbolic address is added to the symbolic address table	
data items are converted into their binary equivalent	
forward references are resolved	

[5]

91. M/J 15/P13/Q5/a,b,c,d

5 (a) Name and describe **three** buses used in the von Neumann model.

Bus 1

Description

Bus 2

Description

Bus 3

Description

[6]

(b) The sequence of operations shows, in register transfer notation, the fetch stage of the fetch-execute cycle.

```
1  MAR ← [PC]
2  PC  ← [PC] + 1
3  MDR ← [ [MAR] ]
4  CIR ← [MDR]
```

- [register] denotes contents of the specified register or memory location
- step 1 above is read as “the contents of the Program Counter are copied to the Memory Address Register”

(i) Describe what is happening at step 2.

[1]

(ii) Describe what is happening at step 3.

[1]

(iii) Describe what is happening at step 4.

[1]

(c) Describe what happens to the registers when the following instruction is executed:

LDD 35

[2]

(d) (i) Explain what is meant by an interrupt.

[2]

(ii) Explain the actions of the processor when an interrupt is detected.

[4]

92. M/J 15/P13/Q7/a,b

7 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load contents of given address to ACC
STO	<address>	Store the contents of ACC at the given address
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
INC	<register>	Add 1 to contents of the register (ACC)
JMP	<address>	Jump to the given address
END		Return control to operating system

The diagram shows the contents of the memory.

Main memory

120	0 0 0 0 1 0 0 1
121	0 1 1 1 0 1 0 1
122	1 0 1 1 0 1 1 0
123	1 1 1 0 0 1 0 0
124	0 1 1 1 1 1 1 1
125	0 0 0 0 0 0 0 1
126	0 1 0 0 0 0 0 1
127	0 1 1 0 1 0 0 1
200	1 0 0 0 1 0 0 0

- (a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 121

Accumulator:

--	--	--	--	--	--	--	--

[1]

- (ii) Show the contents of the Accumulator after execution of the instruction:

LDI 124

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

[3]

(iii) Show the contents of the Accumulator after execution of the instruction:

LDX 120

Index Register:

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

[3]

(b) Trace the assembly language program using the trace table.

300	LDD	321
301	INC	
302	STO	323
303	LDI	307
304	INC	
305	STO	322
306	END	
307	320	
320	49	
321	36	
322	0	
323	0	

Trace table:

Accumulator	Memory address			
	320	321	322	323
	49	36	0	0

[6]

Answers Section

1. M/J 26/P11/Q3/a,b,c

3(a)	1 mark for each correct row <table border="1" data-bbox="236 349 1018 674"> <thead> <tr> <th>accumulator current data</th><th>set of instructions</th><th>new accumulator data</th></tr> </thead> <tbody> <tr> <td>B11001010</td><td>ADD B00000010 INC ACC</td><td>B11001101</td></tr> <tr> <td>#100</td><td>LDM #200 ADD 200</td><td>#301</td></tr> <tr> <td>#103</td><td>LDD 103 DEC ACC DEC ACC</td><td>#48</td></tr> </tbody> </table>	accumulator current data	set of instructions	new accumulator data	B11001010	ADD B00000010 INC ACC	B11001101	#100	LDM #200 ADD 200	#301	#103	LDD 103 DEC ACC DEC ACC	#48	3 Correct answers only
accumulator current data	set of instructions	new accumulator data												
B11001010	ADD B00000010 INC ACC	B11001101												
#100	LDM #200 ADD 200	#301												
#103	LDD 103 DEC ACC DEC ACC	#48												
3(b)(i)	1 mark for AND 1 mark for #0 // &0 // B00000000 Answer: AND B00000000 // AND &0 // AND #0	2 Accept correct shift instructions Bit manipulation so no load commands												
3(b)(ii)	1 mark for the correct answer 0000 0110	1 Correct answer only												
3(c)	1 mark for a correct name 1 mark for a matching description 1 mark for a reasonable example - Indirect - the operand stores the address where the data is found // the data is held in the address stored in the given address - example: LDI 23 loads the data in the address stored in address 23 - Indexed - the address is calculated by adding the operand to the contents of the index register - example: LDX 20 adds 20 to the contents of the index register to calculate the address	3 Do not accept Immediate, relative, direct, absolute, symbolic.												

2. M/J 26/P13/Q7/a,b,c

7(a)	1 mark for each correct answer	4	No mnemonics / opcodes										
	<table><tr><th>Mode of addressing</th><th>Description</th></tr><tr><td>Direct</td><td>The operand is an address where the data is stored.</td></tr><tr><td>Indirect</td><td>The operand is an address that stores the address where the data is stored.</td></tr><tr><td>Indexed</td><td>The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.</td></tr><tr><td>Immediate</td><td>The operand is the data to be used.</td></tr></table>			Mode of addressing	Description	Direct	The operand is an address where the data is stored.	Indirect	The operand is an address that stores the address where the data is stored.	Indexed	The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.	Immediate	The operand is the data to be used.
	Mode of addressing			Description									
	Direct			The operand is an address where the data is stored.									
	Indirect			The operand is an address that stores the address where the data is stored.									
	Indexed			The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.									
Immediate	The operand is the data to be used.												

7(b)	1 mark each to max 3 - Bit manipulation allows a change/evaluation of the state/value of a bit // masking Monitoring: - When monitoring, bit manipulation can be used to establish the state of a device (read inputs) ... by testing/observing the state of an individual bit Controlling: - When controlling bit manipulation can be used to change the state of a device (control output) ... by setting/clearing/toggling a single bit ... or by clearing a byte	3	
7(c)	1 mark for similarity: - Both lose the Least Significant Bit(s) (LSB) // both lose bit(s) from the right of the number 1 mark for difference: - Arithmetic retains sign/MSB while logical does not retain sign/MSB - MSBs are filled with 0 for logical and copies of the sign bit for arithmetic.	2	For difference, must have both sides

3. O/N 25/P11/Q4/a,b,c,d

4(a)	1 mark for: LSL #2	1								
4(b)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	1	1	1	1	0	0	0	1	1
1	1	1	1	0	0	0	1			
4(c)	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	0	0	1	0	0	0	1	1	1
0	0	1	0	0	0	1	1			
4(d)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	0	1	1	1
1	1	1	1	0	0	1	1			

4. O/N 25/P11/Q6/a,b

6(a)	1 mark for each correct description, max 2 marks	2						
	<table><tr><th>Register</th><th>Role</th></tr><tr><td>ACC</td><td>stores the intermediate results of arithmetic and logical operations // holds the result of a calculation</td></tr><tr><td>CIR</td><td>holds the instruction currently being decoded and/or executed</td></tr></table>	Register	Role	ACC	stores the intermediate results of arithmetic and logical operations // holds the result of a calculation	CIR	holds the instruction currently being decoded and/or executed	
Register	Role							
ACC	stores the intermediate results of arithmetic and logical operations // holds the result of a calculation							
CIR	holds the instruction currently being decoded and/or executed							

6(b)	1 mark per bullet point, max 2 marks e.g. • Latency may be increased • ... because the cores must communicate with one another • There is a potential for dead-lock situations • ... where one core may wait for information from other cores which in turn are waiting for the first one • Not all software is designed to use multi-cores • ... so some of the additional cores would be idle • Increased heat generation • ... which could cause damage to other components	2
------	--	----------

5. O/N 25/P12/Q3/a,b,c

3(a)(i)	1 mark The value of the operand is an offset value which is added to another base value to give the address from which the contents are loaded to the accumulator	1								
3(a)(ii)	1 mark per bullet point, max 2 marks - Program counter (PC) - Memory Data Register (MDR) - Memory Address Register (MAR) - Current Instruction Register (CIR) - Status Register	2								
3(b)	1 mark for each correct value, max 3 marks <table><tr><th>Instruction</th><th>Value in ACC</th></tr><tr><td>LDM #98</td><td>98</td></tr><tr><td>LDI 101</td><td>8</td></tr><tr><td>LDX 100</td><td>32</td></tr></table>	Instruction	Value in ACC	LDM #98	98	LDI 101	8	LDX 100	32	3
Instruction	Value in ACC									
LDM #98	98									
LDI 101	8									
LDX 100	32									
3(c)	1 mark per bullet point, max 2 marks for each Clock Speed: - Processor can perform more F-E cycles per second ... so more instructions / data can be processed each second Cache Memory: - Can store more of the most frequently used instructions ... which reduces the need to access slower RAM	4								

6. O/N 25/P13/Q2/c

2(c)	1 mark 1111 0010	1
------	--------------------------------	----------

7. O/N 25/P13/Q6/a,b,c

6(a)	1 mark for each correct row <table border="1"> <thead> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>Contents of the ACC after the execution of the instruction</th></tr> </thead> <tbody> <tr> <td>0000 1111</td><td>AND 101</td><td>0000 1110</td></tr> <tr> <td>0000 0000</td><td>LDM #100</td><td>0110 0100</td></tr> <tr> <td>0000 0001</td><td>XOR &F1</td><td>1111 0000</td></tr> <tr> <td>0001 0001</td><td>CMP 101</td><td>0001 0001</td></tr> </tbody> </table>	Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction	0000 1111	AND 101	0000 1110	0000 0000	LDM #100	0110 0100	0000 0001	XOR &F1	1111 0000	0001 0001	CMP 101	0001 0001	4
Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction															
0000 1111	AND 101	0000 1110															
0000 0000	LDM #100	0110 0100															
0000 0001	XOR &F1	1111 0000															
0001 0001	CMP 101	0001 0001															
6(b)	1 mark for each bullet point, max 4 marks - MAR stores the <u>address</u> of the next instruction / data to be read from / or written to memory - The address is received from the Program Counter (PC) - The MDR stores the data / instruction in the address stored in the MAR which has been read / written - The instruction passes to the CIR for decoding and executing	4															
6(c)	1 mark for each bullet point, max 3 marks - Data movement - Arithmetic operations - Conditional and unconditional (jump) instructions	3															

8. M/J 25/P11/Q6/a(ii,iii)

6(a)(ii)	1 mark each to max 3 - More RAM means more currently running data and instructions can be stored - without needing to use virtual memory - without having to fetch the data from secondary storage first - which has a slower access time - Less latency / delay waiting for instructions / data	3
----------	--	----------

6(a)(iii)	1 mark each to max 2 e.g. - Wider data bus means more data can be transferred between components at a time - there is less delay / latency when fetching data for a running process - Wider address bus means larger memory addresses can be used - allowing more memory locations to be accessed directly - so less likely to run out of memory	2
-----------	---	----------

9. M/J 25/P11/Q8/a,b

8(a)	1 mark for each shaded section	4																																																																						
	<table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th></tr><tr><th>80</th><th>81</th><th>82</th><th>83</th></tr><tr><td></td><td></td><td>10</td><td>8</td><td>80</td><td>81</td></tr><tr><td>200</td><td>8</td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td>9</td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td>9</td></tr><tr><td>203</td><td>10</td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>206</td><td>9</td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td>19</td><td></td><td></td><td></td><td></td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>210</td><td></td><td></td><td>19</td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				80	81	82	83			10	8	80	81	200	8					201	9					202					9	203	10					204						206	9					207	19					208						210			19			
Instruction address	ACC			Memory address																																																																				
		80	81	82	83																																																																			
		10	8	80	81																																																																			
200	8																																																																							
201	9																																																																							
202					9																																																																			
203	10																																																																							
204																																																																								
206	9																																																																							
207	19																																																																							
208																																																																								
210			19																																																																					
8(b)(i)	1 mark for: OR B0000 0001 // OR #1 // OR &1	1																																																																						
8(b)(ii)	1 mark for: 1010 1011	1																																																																						
8(b)(iii)	1 mark for: 0000 0011	1																																																																						

10. M/J 25/P12/Q1/a,b,c

1(a)	1 mark for each correct row <table><tr><th>Incorrect statement number</th><th>Corrected statement</th></tr><tr><td>1</td><td>The Program Counter (PC) stores the address of the next instruction to be fetched from memory.</td></tr><tr><td>3</td><td>The Control Unit (CU) sends signals to other components on the control bus.</td></tr><tr><td>4</td><td>The Memory Data Register (MDR) holds data to be stored in the memory address in the MAR. // The Memory Data Register (MDR) holds data read from the memory address in the MAR</td></tr></table>	Incorrect statement number	Corrected statement	1	The Program Counter (PC) stores the address of the next instruction to be fetched from memory.	3	The Control Unit (CU) sends signals to other components on the control bus.	4	The Memory Data Register (MDR) holds data to be stored in the memory address in the MAR. // The Memory Data Register (MDR) holds data read from the memory address in the MAR	3
Incorrect statement number	Corrected statement									
1	The Program Counter (PC) stores the address of the next instruction to be fetched from memory.									
3	The Control Unit (CU) sends signals to other components on the control bus.									
4	The Memory Data Register (MDR) holds data to be stored in the memory address in the MAR. // The Memory Data Register (MDR) holds data read from the memory address in the MAR									
1(b)	1 mark for name, 1 mark for corresponding role e.g. • Current Instruction Register // CIR • To store the instruction to be decoded / executed next • Status register // SR • To contain bits that can be referenced individually to indicate a state or event • Interrupt register • To store details of any interrupts that have occurred	2								
1(c)	1 mark each to max 4 • At the start / end of FE cycle the interrupt register is checked • The priority of any interrupts waiting is checked • if the priority of the interrupt is higher than the current process • the contents of the registers are stored on the stack • The relevant Interrupt Service Routine (ISR) / interrupt handler is called to process the interrupt • When the ISR has finished, a further check is made for higher priority interrupts • if no more interrupts of higher priority, the register contents are restored and the next FE cycle continues	4								

11. M/J 25/P13/Q2/a,b,c

2(a)(i)	1 mark each to max 2 - max 1 for each set of marks • The control unit synchronises the actions of the processor • by sending a command / signal on each timing signal produced by the system clock • using / along the control bus	2
---------	---	----------

2(a)(ii)	1 mark each • $MAR \leftarrow [PC]$ and $PC \leftarrow [PC] + 1$ • $MDR \leftarrow [[MAR]]$ • $CIR \leftarrow [MDR]$ • Correct order • For example: • $MAR \leftarrow [PC]$ • $PC \leftarrow [PC] + 1$ • $MDR \leftarrow [[MAR]]$ • $CIR \leftarrow [MDR]$	4
2(b)	1 mark each • Using cache memory improves system performance • because cache is fast access memory close to the CPU • which stores frequently used instructions / data • so that they can be accessed faster than from RAM	2
2(c)	1 mark each to max 2 e.g. • HDMI transfers both audio and video using a single cable • HDMI has a high bandwidth • Data is transmitted in a stream • of uncompressed digital signals • HDMI uses a technology called Transition-Minimized Differential Signalling (TMDS)	2

12. M/J 25/P13/Q5/a,b

5(a)(i)	1 mark for each shaded section <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th></tr><tr><th>50</th><th>51</th><th>52</th><th>53</th></tr><tr><td></td><td></td><td>47</td><td>48</td><td>49</td><td>50</td></tr><tr><td>500</td><td>50</td><td></td><td></td><td></td><td></td></tr><tr><td>501</td><td>49</td><td></td><td></td><td></td><td></td></tr><tr><td>502</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>504</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>501</td><td>48</td><td></td><td></td><td></td><td></td></tr><tr><td>502</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>503</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>505</td><td></td><td>48</td><td></td><td></td><td></td></tr><tr><td>506</td><td>38</td><td></td><td></td><td></td><td></td></tr><tr><td>507</td><td></td><td></td><td>38</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				50	51	52	53			47	48	49	50	500	50					501	49					502						504						501	48					502						503						505		48				506	38					507			38															3
Instruction address	ACC			Memory address																																																																																						
		50	51	52	53																																																																																					
		47	48	49	50																																																																																					
500	50																																																																																									
501	49																																																																																									
502																																																																																										
504																																																																																										
501	48																																																																																									
502																																																																																										
503																																																																																										
505		48																																																																																								
506	38																																																																																									
507			38																																																																																							
5(a)(ii)	1 mark for addressing mode, 1 mark for matching description • Direct • The operand is the address of the data • Indirect • The operand points to the memory location which is the address of the data • Indexed • The address of the data is formed by adding the contents of the Index Register (IX) to the operand	4																																																																																								
5(b)	1 mark for each correct instruction e.g. AND B00000000 / #0 / &0 OR B10000000 / #128 / &80	2																																																																																								

13. M/J 25/P13/Q7/a

7(a)(i)	1 mark for 0010 1000	1
7(a)(ii)	1 mark for 1111 0011	1

14. O/N 24/P11/Q3/a,b

3(a)	1 mark for system clock and 1 mark for Control Unit System clock • To synchronise operations • ... by creating and transmitting timing signals on the control bus Control Unit • Sends/receives control signals along control bus • Reads an instruction from the contents of the memory location whose address is stored in PC • Coordinates/synchronises the activity of other components in the CPU • Manages the execution of instructions • Controls communication between the components in the CPU	2
3(b)(i)	1 mark for the feature, 1 mark max for the matching reason e.g. Feature: clock speed • Higher clock speed means that more F-E cycles are executed per second // Higher clock speed results in more throughput Feature: bus width • Larger bus width means that more data transferred at the same time	2
3(b)(ii)	1 mark for correct port. 2 marks for explanation Port: USB / Universal Serial Bus Explanation: • A voltage change occurs when the drive is plugged in • The computer detects this voltage change • The code of the device is transferred to computer • ... the OS finds the code of the device in the list of devices • ... and loads the appropriate device driver	3

15. O/N 24/P11/Q8/a,b

8(a)	1 mark for each bullet point (max 2) - To store the value of flags/bits ... that can be changed/set/cleared after arithmetic / logical operations - To allow flags to be checked ... to change instruction sequence	2
8(b)	1 mark for each bullet point (max 2) - Special purpose registers have a specified role in the machine whereas general purpose registers can be used for all purposes defined by the programmer - Special purpose registers hold the state of the program's execution while general purpose registers hold the program's data during operations	2

16. O/N 24/P12/Q3/a,b

3(a)	1 mark for each correct purpose	4										
<table><tr><th>Register</th><th>Purpose</th></tr><tr><td>Program Counter (PC)</td><td>Stores the address of the next instruction to be fetched/executed</td></tr><tr><td>Memory Address Register (MAR)</td><td>Stores the address of the memory location where data will be read from/written to</td></tr><tr><td>Memory Data Register (MDR)</td><td>Stores the data read from the address in the MAR // stores the data to be written to the address in the MAR</td></tr><tr><td>Index Register (IX)</td><td>Stores a number that will be added to the operand, to form the address of the data</td></tr></table>			Register	Purpose	Program Counter (PC)	Stores the address of the next instruction to be fetched/executed	Memory Address Register (MAR)	Stores the address of the memory location where data will be read from/written to	Memory Data Register (MDR)	Stores the data read from the address in the MAR // stores the data to be written to the address in the MAR	Index Register (IX)	Stores a number that will be added to the operand, to form the address of the data
Register	Purpose											
Program Counter (PC)	Stores the address of the next instruction to be fetched/executed											
Memory Address Register (MAR)	Stores the address of the memory location where data will be read from/written to											
Memory Data Register (MDR)	Stores the data read from the address in the MAR // stores the data to be written to the address in the MAR											
Index Register (IX)	Stores a number that will be added to the operand, to form the address of the data											
3(b)	1 mark for each bullet point (max 4) e.g. • HDMI has faster transfer rates than VGA • ... needed due to high resolution / large number of pixels of monitor // HDMI supports the high resolution of the monitor • HDMI supports video and audio transfer between computer and monitor speakers • ... so no separate sound cable is needed unlike VGA • HDMI is digital interface therefore no data is lost in transfer to analogue and back • HDMI is less prone to error/crosstalk/external interference	4										

17. O/N 24/P12/Q8/a,b

8(a)	1 mark for each correct content of the ACC (4) 1 mark for correct IX column <table><tr><td></td><td>Instructions</td><td>ACC content</td><td>IX content</td></tr><tr><td>1</td><td>LDM #19 DEC ACC</td><td>18</td><td>20</td></tr><tr><td>2</td><td>LDD 23 ADD 19</td><td>40</td><td>20</td></tr><tr><td>3</td><td>LDI 25 INC ACC</td><td>5</td><td>20</td></tr><tr><td>4</td><td>LDR #21 LDX 2</td><td>15</td><td>21</td></tr></table>		Instructions	ACC content	IX content	1	LDM #19 DEC ACC	18	20	2	LDD 23 ADD 19	40	20	3	LDI 25 INC ACC	5	20	4	LDR #21 LDX 2	15	21	5
	Instructions	ACC content	IX content																			
1	LDM #19 DEC ACC	18	20																			
2	LDD 23 ADD 19	40	20																			
3	LDI 25 INC ACC	5	20																			
4	LDR #21 LDX 2	15	21																			
8(b)(i)	1 mark each correct content of ACC <table><tr><td></td><td>Instructions</td><td>ACC content</td></tr><tr><td>1</td><td>LSL #2</td><td>0110 1000</td></tr><tr><td>2</td><td>ADD #5 AND #30</td><td>0001 1110</td></tr><tr><td>3</td><td>OR B11110010 INC ACC</td><td>1111 1011</td></tr></table>		Instructions	ACC content	1	LSL #2	0110 1000	2	ADD #5 AND #30	0001 1110	3	OR B11110010 INC ACC	1111 1011	3								
	Instructions	ACC content																				
1	LSL #2	0110 1000																				
2	ADD #5 AND #30	0001 1110																				
3	OR B11110010 INC ACC	1111 1011																				
8(b)(ii)	1 mark each to max 2 for explanation - An odd binary number will have a 1 in the Least Significant Bit (LSB) - A bit manipulation operation is required to access/mask only the LSB and clear all the others - Compare the result of the masking with denary 1 ... the result of the comparison will be true if the number is odd 1 mark for correct instruction - AND B00000001 // AND #1 // AND &01	3																				

18. O/N 24/P13/Q7/a,b

7(a)	1 mark each: <table border="1" data-bbox="284 282 1046 730"> <thead> <tr> <th></th><th>Instructions</th><th>ACC content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDD 50 ADD #4 ADD 54</td><td>158</td></tr> <tr> <td>2</td><td>LDI 53 DEC ACC ADD 56</td><td>99</td></tr> <tr> <td>3</td><td>LDM #55 SUB #5</td><td>50</td></tr> </tbody> </table>		Instructions	ACC content	1	LDD 50 ADD #4 ADD 54	158	2	LDI 53 DEC ACC ADD 56	99	3	LDM #55 SUB #5	50	3
	Instructions	ACC content												
1	LDD 50 ADD #4 ADD 54	158												
2	LDI 53 DEC ACC ADD 56	99												
3	LDM #55 SUB #5	50												
7(b)	1 mark for each bullet point for the explanation 1 mark for correct instruction • A bit manipulation operation is required to set all the bits to zero • Compare the result of the masking with 0 • ... the result of comparison will be true if the register is cleared • AND B00000000 / #00 / &00	3												

19. O/N 24/P13/Q8/c

8(c)	1 mark each: • A logical shift moves all bits to the right and inserts zeros in the appropriate leftmost bits • An arithmetic shift moves all bits to the right but copies the sign bit into the Most Significant Bit (MSB)	2
------	---	----------

20. M/J 24/P11/Q4/a,b

4(a)	1 mark for each correct answer: <table border="1" data-bbox="284 1552 861 2074"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDD 20 ADD #2</td><td>4</td></tr> <tr> <td>2</td><td>LDX 22</td><td>5</td></tr> <tr> <td>3</td><td>LDI 25 INC ACC SUB 22</td><td>1</td></tr> <tr> <td>4</td><td>LDD 19 LDM #5 LDM #25</td><td>25</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDD 20 ADD #2	4	2	LDX 22	5	3	LDI 25 INC ACC SUB 22	1	4	LDD 19 LDM #5 LDM #25	25	4
Program Number	Code	ACC Content															
1	LDD 20 ADD #2	4															
2	LDX 22	5															
3	LDI 25 INC ACC SUB 22	1															
4	LDD 19 LDM #5 LDM #25	25															

4(b)	1 mark for each correct answer: <table border="1" data-bbox="284 237 951 533"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>AND 31</td><td>1001 1010</td></tr> <tr> <td>2</td><td>XOR B01001111</td><td>1101 0101</td></tr> <tr> <td>3</td><td>OR #30</td><td>1001 1110</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	AND 31	1001 1010	2	XOR B01001111	1101 0101	3	OR #30	1001 1110	3
Program Number	Code	ACC Content												
1	AND 31	1001 1010												
2	XOR B01001111	1101 0101												
3	OR #30	1001 1110												

21. M/J 24/P12/Q5/a,b

5(a)	1 mark for each correct answer: <table border="1" data-bbox="284 714 1123 1279"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDI 15 SUB #1</td><td>11</td></tr> <tr> <td>2</td><td>LDD 14 ADD 11</td><td>13</td></tr> <tr> <td>3</td><td>LDM #11 ADD #3 SUB 16</td><td>2</td></tr> <tr> <td>4</td><td>LDR #2 LDX 14 ADD #2</td><td>14</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDI 15 SUB #1	11	2	LDD 14 ADD 11	13	3	LDM #11 ADD #3 SUB 16	2	4	LDR #2 LDX 14 ADD #2	14	4
Program Number	Code	ACC Content															
1	LDI 15 SUB #1	11															
2	LDD 14 ADD 11	13															
3	LDM #11 ADD #3 SUB 16	2															
4	LDR #2 LDX 14 ADD #2	14															
5(b)	1 mark for each correct answer: <table border="1" data-bbox="284 1386 1123 1686"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>XOR 29</td><td>0100 1001</td></tr> <tr> <td>2</td><td>AND #29</td><td>0000 0100</td></tr> <tr> <td>3</td><td>OR B11111111</td><td>1111 1111</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	XOR 29	0100 1001	2	AND #29	0000 0100	3	OR B11111111	1111 1111	3			
Program Number	Code	ACC Content															
1	XOR 29	0100 1001															
2	AND #29	0000 0100															
3	OR B11111111	1111 1111															

22. M/J 24/P13/Q3/a,b

3(a)	1 mark for each correct answer: <table border="1" data-bbox="284 282 844 873"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDM #50 INC ACC SUB #1</td><td>50</td></tr> <tr> <td>2</td><td>LDI 51 ADD 52</td><td>97</td></tr> <tr> <td>3</td><td>LDR #2 LDX 50 DEC ACC</td><td>48</td></tr> <tr> <td>4</td><td>LDD 52 SUB 54 INC ACC</td><td>44</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDM #50 INC ACC SUB #1	50	2	LDI 51 ADD 52	97	3	LDR #2 LDX 50 DEC ACC	48	4	LDD 52 SUB 54 INC ACC	44	4
Program Number	Code	ACC Content															
1	LDM #50 INC ACC SUB #1	50															
2	LDI 51 ADD 52	97															
3	LDR #2 LDX 50 DEC ACC	48															
4	LDD 52 SUB 54 INC ACC	44															
3(b)	1 mark for each correct answer: <table border="1" data-bbox="284 983 1008 1279"> <thead> <tr> <th>Instruction Number</th><th>Instruction</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LSL #2</td><td>1111 1100</td></tr> <tr> <td>2</td><td>XOR 100</td><td>1111 0010</td></tr> <tr> <td>3</td><td>AND 103</td><td>0011 0111</td></tr> </tbody> </table>	Instruction Number	Instruction	ACC Content	1	LSL #2	1111 1100	2	XOR 100	1111 0010	3	AND 103	0011 0111	3			
Instruction Number	Instruction	ACC Content															
1	LSL #2	1111 1100															
2	XOR 100	1111 0010															
3	AND 103	0011 0111															

23. O/N 23/P11/Q5/a,b,c

5(a)	1 mark for identification of the register and 1 mark for role (max 2 for each register) • Program Counter (PC) • stores the <u>address</u> where the <u>next</u> instruction is to be read from • Memory Address Register (MAR) • stores the address of the memory location (or an I/O component) currently being read from or written to • Current Instruction Register (CIR) • holds the instruction currently being decoded and/or executed • Status Register • Contains bits which can be referenced individually and set or cleared depending on the operation e.g. overflow, underflow	4
------	--	----------

5(b)	1 mark for each bullet point (max 2) - Immediate Access Store holds all the data / instructions / programs currently in use - Immediate Access Store is volatile memory - Immediate Access Store has fast access times	2
5(c)(i)	1 mark for each bullet point (max 1) - The CPU can now perform nearly twice as many F-E cycles per second - Instead of 2.1 billion F-E cycles per second, the CPU can now perform 4 billion FE cycles per second	1
5(c)(ii)	1 mark for each bullet point (max 5) - Multiple cores introduce additional overheads ...because of the need for communication between cores - Software may not be designed for multiple cores... ...so one of the cores will be left idle - Memory access speed may not match speed of cores... ...so causing delay - The two computers may have more differences than just the cores ...one may have more RAM which allows faster multitasking ...one may have a GPU ...etc.	5

24. O/N 23/P11/Q8/a,b,c

8(a)	1 mark for: To create a symbol table	1
8(b)(i)	1 mark for each bullet point - Data movement: e.g. LDR #50 // STO 201 - Arithmetic operation: e.g. ADD 100 // INC IX - Conditional instruction: e.g. JPE 96	3
8(b)(ii)	1 mark for each bullet point (max 2) Similarity: - both load the contents of an <u>address</u> into the <u>Accumulator</u> Difference: - direct accesses the address given by the operand whereas indexed adds the contents of IX to the operand and accesses the data at that calculated address	2
8(b)(iii)	1 mark for - Indirect (addressing) - Relative (addressing)	1
8(c)(i)	0000 0101	1

8(c)(ii)	0000 0010	1
8(c)(iii)	1101 0010	1

25. O/N 23/P12/Q8/c

8(c)(i)	1 mark for each bullet point (max 1) • USB / Universal Serial Bus • HDMI	1
8(c)(ii)	1 mark for each component (max 3) Buffers • A buffer temporarily holds data until it is ready to be transmitted to the device Address Bus • The address of the data to be written to the device (in RAM) is carried on the address bus Data Bus • All data to be written to the device / buffer is carried on the data bus	3

26. O/N 23/P12/Q9/a,b

9(a)	1 mark for each bullet point • IN - Input and output of data • ADD - Arithmetic operations • JPE - Unconditional and conditional instructions • CMI - Compare instructions	4
------	---	----------

9(b)

1 mark for each set of shaded rows

4

Instruction address	ACC	IX	Memory address				Output
			100	101	110	111	
			0	0	51	65	
10		0					
11	49						
12				49			
13	51						
14							
15							
16	49						
17	65						
18		1					
19			65				
20							
13	65						
14							
15							
21							A
22							

27. O/N 23/P13/Q7/d

7(d)	1 mark for naming a correct port 2 marks for matching justification	3
	• USB • ...has fast data transfer speeds for data (to the headset) • ...is a universal/popular cable // universal standard • HDMI • ...allows video and audio to be transferred (on the same cable) • ...convenience of HDMI as no need for two cables	

28. O/N 23/P13/Q9/a,b,c,d

9(a)(i)	1 mark for each bullet point (max 3).	3
	• Control unit (CU) • Arithmetic and Logic Unit (ALU) • Immediate Access Store (IAS) • (System clock)	

9(a)(ii)	1 mark for each bullet point (max 2). - Special purpose registers hold the status of a program whereas general purpose registers hold the temporary data while performing operations. - Special purpose registers are specialised for a specific use, whereas general purpose registers are used for any purpose. - General purpose registers can be used by most instructions, whereas special purpose can only be used by certain instruction	2										
9(b)	1 mark for each correct answer (shaded cells) <table><tr><th>Step</th><th>Description</th></tr><tr><td>PC ← [PC] + 1</td><td>The address in PC is incremented.</td></tr><tr><td>MDR ← [[MAR]]</td><td>The data in the location pointed to by the MAR is copied to the MDR.</td></tr><tr><td>MAR ← [PC]</td><td>The contents of PC are copied to the MAR.</td></tr><tr><td>CIR ← [MDR]</td><td>The contents of MDR are copied into CIR.</td></tr></table>	Step	Description	PC ← [PC] + 1	The address in PC is incremented.	MDR ← [[MAR]]	The data in the location pointed to by the MAR is copied to the MDR.	MAR ← [PC]	The contents of PC are copied to the MAR.	CIR ← [MDR]	The contents of MDR are copied into CIR.	4
Step	Description											
PC ← [PC] + 1	The address in PC is incremented.											
MDR ← [[MAR]]	The data in the location pointed to by the MAR is copied to the MDR.											
MAR ← [PC]	The contents of PC are copied to the MAR.											
CIR ← [MDR]	The contents of MDR are copied into CIR.											
9(c)	1 mark for an appropriate example. Examples: - Division by zero - Runtime error - Out of memory bounds - Program requesting an external device / input - Buffer overflow	1										
9(d)	1 mark for each letter in the correct position (2, 4 and 6) D A B	3										

29. M/J 23/P11/Q3/d(vi)

3(d)(vi)	00011001	1
----------	----------	----------

30. M/J 23/P11/Q6

6	1 mark each to max 5 • An interrupt flag is raised in the (interrupt) register • At the end of the current FE cycle // at the start of the next FE cycle • The system checks the interrupt register for higher priority interrupts than current process • If true, it stores the current contents of the registers on the stack • The appropriate interrupt service routine (ISR) for the key press is called • The input data from the keyboard is processed • The contents of the registers are restored from the stack • ... and control is passed back to previous process	5
----------	---	----------

31. M/J 23/P12/Q3/a,b

3(a)	1 mark for generates object code to second pass 1 mark for reads source code one line at a time to both boxes 1 mark for removes white space and adds labels to first pass <table><thead><tr><th>Action</th><th>Pass</th></tr></thead><tbody><tr><td>generates object code</td><td>second</td></tr><tr><td>reads the source code one line at a time</td><td>first</td></tr><tr><td>reads the source code one line at a time</td><td>second</td></tr><tr><td>removes white space</td><td>first</td></tr><tr><td>adds labels to the symbol table</td><td>first</td></tr></tbody></table>	Action	Pass	generates object code	second	reads the source code one line at a time	first	reads the source code one line at a time	second	removes white space	first	adds labels to the symbol table	first	3
Action	Pass													
generates object code	second													
reads the source code one line at a time	first													
reads the source code one line at a time	second													
removes white space	first													
adds labels to the symbol table	first													
3(b)	1 mark for each correct term Direct addressing is when the operand holds the memory address of the data. Indirect addressing is when the operand holds a memory address that stores the memory address of the data. Immediate addressing is when the operand is the data.	3												

32. M/J 23/P12/Q4/d

4(d)	0011 1100	1
------	-----------	----------

33. M/J 23/P12/Q5/a,b,c

5(a)	1 mark each to max 2 Examples: • Interrupt • Timing • Read • Write	2
5(b)	1 mark for description; 1 mark for corresponding explanation Examples • Increase number of cores • Each core can independently carry out a process at the same time // so that more instructions are performed in parallel • Increase RAM capacity • ... allowing more applications to reside in memory at the same time, saving disk access times • Increase cache memory • More data can be stored in fast access so less time is spent accessing from RAM • Increase clock speed • More Fetch-Decode-Execute (FDE) cycles can run each second / per unit time	4
5(c)(i)	1 mark for a correct answer • 1 bit is transferred at a time • Can be synchronous or asynchronous • USB-3 is full duplex and earlier versions are half-duplex	1
5(c)(ii)	1 mark for identification of a suitable port Examples • HDMI • DisplayPort	1

34. M/J 23/P13/Q7/b,c

7(b)	1 mark each to max 4 • The system clock gives out timing signals • ... which are sent on the control bus • ...to synchronise the other system components • The Control Unit initiates data transfer • ...by generating signals that are sent on the control bus to other components	4						
7(c)	1 mark for each Register transfer notation <table><tr><th>Stage description</th><th>Register transfer notation</th></tr><tr><td>The Program Counter (PC) is incremented</td><td>$PC \leftarrow [PC] + 1$</td></tr><tr><td>The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)</td><td>$MDR \leftarrow [[MAR]]$</td></tr></table>	Stage description	Register transfer notation	The Program Counter (PC) is incremented	$PC \leftarrow [PC] + 1$	The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)	$MDR \leftarrow [[MAR]]$	2
Stage description	Register transfer notation							
The Program Counter (PC) is incremented	$PC \leftarrow [PC] + 1$							
The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)	$MDR \leftarrow [[MAR]]$							

35. O/N 22/P11/Q5/a,b,c,d

5(a)	Instructions and data are stored in the same memory space / in main memory.	1												
5(b)(i)	1 mark for each special purpose register: Program Counter (PC): - to store the address / location / memory location of the <u>next</u> instruction to be fetched Index Register (IX): - to store a value that is added to an address to give another address Status Register (SR): - to store flags which are set by events // from the results of arithmetic and logic operations and interrupt flags	3												
5(b)(ii)	1 mark for both rows: <table><tr><th>CPU component</th><th>Data bus</th><th>Address bus</th><th>Control bus</th></tr><tr><td>System clock</td><td></td><td></td><td>✓</td></tr><tr><td>Memory Address Register (MAR)</td><td></td><td>✓</td><td></td></tr></table>	CPU component	Data bus	Address bus	Control bus	System clock			✓	Memory Address Register (MAR)		✓		1
CPU component	Data bus	Address bus	Control bus											
System clock			✓											
Memory Address Register (MAR)		✓												

5(b)(iii)	1 mark for each bullet point (max 2): • to coordinate / synchronise the actions of other components in the CPU • to send / receive control signals along the control bus • to manage the execution of instructions (in sequence) • to control the communication between the components of the CPU	2
5(c)	1 mark for each bullet point: • to send a signal from a device or process • ... seeking the attention of the processor	2
5(d)	1 mark for each bullet point (max 2). For example: • division by zero // runtime error in a program • attempt to access an invalid memory location • array index out of bounds • stack overflow	2

36. O/N 22/P11/Q6/a,b,c

6(a)(i)	1 mark for each set of highlighted rows. <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th rowspan="2">IX</th><th colspan="5">Memory address</th><th rowspan="2">Output</th></tr><tr><th>100</th><th>101</th><th>110</th><th>111</th><th>112</th></tr><tr><td></td><td></td><td></td><td>0</td><td>0</td><td>66</td><td>65</td><td>35</td><td></td></tr><tr><td>77</td><td></td><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>79</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td>66</td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>35</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>90</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>91</td><td>50</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>92</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>2</td></tr><tr><td>93</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	IX	Memory address					Output	100	101	110	111	112				0	0	66	65	35		77		0							78	66								79									80									81									82				66					83	1								84									85			1						86		1							87	65								88									89									81	66								82									83	1								84	2								85			2						86		2							87	35								88									89									90	2								91	50								92								2	93									4
Instruction address	ACC				IX	Memory address					Output																																																																																																																																																																																																																																																								
		100	101	110		111	112																																																																																																																																																																																																																																																												
			0	0	66	65	35																																																																																																																																																																																																																																																												
77		0																																																																																																																																																																																																																																																																	
78	66																																																																																																																																																																																																																																																																		
79																																																																																																																																																																																																																																																																			
80																																																																																																																																																																																																																																																																			
81																																																																																																																																																																																																																																																																			
82				66																																																																																																																																																																																																																																																															
83	1																																																																																																																																																																																																																																																																		
84																																																																																																																																																																																																																																																																			
85			1																																																																																																																																																																																																																																																																
86		1																																																																																																																																																																																																																																																																	
87	65																																																																																																																																																																																																																																																																		
88																																																																																																																																																																																																																																																																			
89																																																																																																																																																																																																																																																																			
81	66																																																																																																																																																																																																																																																																		
82																																																																																																																																																																																																																																																																			
83	1																																																																																																																																																																																																																																																																		
84	2																																																																																																																																																																																																																																																																		
85			2																																																																																																																																																																																																																																																																
86		2																																																																																																																																																																																																																																																																	
87	35																																																																																																																																																																																																																																																																		
88																																																																																																																																																																																																																																																																			
89																																																																																																																																																																																																																																																																			
90	2																																																																																																																																																																																																																																																																		
91	50																																																																																																																																																																																																																																																																		
92								2																																																																																																																																																																																																																																																											
93																																																																																																																																																																																																																																																																			
6(a)(ii)	swaps the contents of memory address <u>100</u> and <u>101</u>	1																																																																																																																																																																																																																																																																	
6(b)(i)	1000 1100	1																																																																																																																																																																																																																																																																	

6(b)(ii)	1001 0000	1															
6(b)(iii)	1101 1111	1															
6(b)(iv)	0010 0100	1															
6(c)	1 mark for each pair of highlighted rows <table> <tr> <th>Task</th><th>First pass</th><th>Second pass</th></tr> <tr> <td>Remove comments.</td><td>✓</td><td></td></tr> <tr> <td>Read the assembly language program one line at a time.</td><td>✓</td><td>✓</td></tr> <tr> <td>Generate the object code.</td><td></td><td>✓</td></tr> <tr> <td>Check the opcode is in the instruction set.</td><td>✓</td><td></td></tr> </table>	Task	First pass	Second pass	Remove comments.	✓		Read the assembly language program one line at a time.	✓	✓	Generate the object code.		✓	Check the opcode is in the instruction set.	✓		2
Task	First pass	Second pass															
Remove comments.	✓																
Read the assembly language program one line at a time.	✓	✓															
Generate the object code.		✓															
Check the opcode is in the instruction set.	✓																

37. O/N 22/P11/Q7/d,e

7(d)	1 mark for each bullet point (max 2): • cache is fast access memory (close to the CPU) • cache stores frequently used instructions / data • ... more cache means more instructions / data can be transferred faster • ... less swapping between RAM and cache • prevents the CPU idling while waiting for data	2
7(e)	1 mark for each device. • 3D printer: USB port / COM port • Monitor: HDMI / VGA / USB / DisplayPort	2

38. O/N 22/P12/Q7/a,b,c

7(a)	1 mark for each set of highlighted rows.										5
</											

7(b)(iii)	1001 1000	1								
7(b)(iv)	1101 1111	1								
7(c)	1 mark for each correct row: <table><thead><tr><th>Description</th><th>Register transfer notation</th></tr></thead><tbody><tr><td>Copy the address of the next instruction into the Memory Address Register.</td><td>MAR ← [PC]</td></tr><tr><td>Increment the Program Counter.</td><td>PC ← [PC] + 1</td></tr><tr><td>Copy the contents of the Memory Data Register into the Current Instruction Register.</td><td>CIR ← [MDR]</td></tr></tbody></table>	Description	Register transfer notation	Copy the address of the next instruction into the Memory Address Register.	MAR ← [PC]	Increment the Program Counter.	PC ← [PC] + 1	Copy the contents of the Memory Data Register into the Current Instruction Register.	CIR ← [MDR]	3
Description	Register transfer notation									
Copy the address of the next instruction into the Memory Address Register.	MAR ← [PC]									
Increment the Program Counter.	PC ← [PC] + 1									
Copy the contents of the Memory Data Register into the Current Instruction Register.	CIR ← [MDR]									

39. O/N 22/P13/Q4/a,b,c

4(a)(i)	1 mark for each register: MAR: holds address in memory from which data will be read / to which data will be written MDR: holds the data/instructions which has been read from or is to be written to the address in the MAR	2
4(a)(ii)	after completion of the execute stage // before the cycle begins	1
4(b)	1 mark for each bullet point (max 2): - synchronise operations ... by creating timing signals - to keep track of the date and time / timestamp files - to process operations in the correct order / sequence	2
4(c)	1 mark for identification of a correct upgrade: 1 mark for a corresponding explanation: Examples: - increase quantity of RAM ... so allowing more applications to reside in memory at the same time, saving disk access times - increase the size of cache memory ... so that the CPU can continue working without waiting for data - increase clock speed ... so that more instructions are performed in a time period - increase the number of processors / cores ... so that more instructions are performed in parallel	2

40. O/N 22/P13/Q6/a,b,c

6(a)(i)	1 mark for each set of highlighted rows: <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th rowspan="2">IX</th><th colspan="5">Memory address</th><th>Output</th></tr><tr><th>100</th><th>101</th><th>110</th><th>111</th><th>112</th><th></th></tr><tr><td></td><td></td><td></td><td>1</td><td>0</td><td>97</td><td>98</td><td>97</td><td></td></tr><tr><td>75</td><td></td><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>77</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>79</td><td>97</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>90</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>77</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>91</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>92</td><td>49</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>93</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td></tr><tr><td>94</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	IX	Memory address					Output	100	101	110	111	112					1	0	97	98	97		75		0							76	1								77									78									79	97								80	65								81									82									83	1								84									85				1					86									87	2								88			2						89		1							90									76									77									78									91	1								92	49								93								1	94									4
Instruction address	ACC				IX	Memory address					Output																																																																																																																																																																																																																														
		100	101	110		111	112																																																																																																																																																																																																																																		
			1	0	97	98	97																																																																																																																																																																																																																																		
75		0																																																																																																																																																																																																																																							
76	1																																																																																																																																																																																																																																								
77																																																																																																																																																																																																																																									
78																																																																																																																																																																																																																																									
79	97																																																																																																																																																																																																																																								
80	65																																																																																																																																																																																																																																								
81																																																																																																																																																																																																																																									
82																																																																																																																																																																																																																																									
83	1																																																																																																																																																																																																																																								
84																																																																																																																																																																																																																																									
85				1																																																																																																																																																																																																																																					
86																																																																																																																																																																																																																																									
87	2																																																																																																																																																																																																																																								
88			2																																																																																																																																																																																																																																						
89		1																																																																																																																																																																																																																																							
90																																																																																																																																																																																																																																									
76																																																																																																																																																																																																																																									
77																																																																																																																																																																																																																																									
78																																																																																																																																																																																																																																									
91	1																																																																																																																																																																																																																																								
92	49																																																																																																																																																																																																																																								
93								1																																																																																																																																																																																																																																	
94																																																																																																																																																																																																																																									
6(a)(ii)	1 mark for each bullet point: To allow for re-locatable code ... because all (target) addresses can be specified by the base address + offset	2																																																																																																																																																																																																																																							
6(b)(i)	0000 0100	1																																																																																																																																																																																																																																							

6(b)(ii)	1101 1111	1
6(b)(iii)	0010 0111	1
6(c)	1 mark for a correct name: - input and output of data - arithmetic operations - unconditional and conditional instructions - compare instructions	1

41. O/N 21/P11/Q6/a,b

6(a)	1 mark for identification of line and description of error 1 mark for the correct statement	4															
	<table> <tr> <th>Line number</th><th>Description of the error</th><th>Correct statement</th></tr> <tr> <td>2</td><td>Program Counter should be incremented, not decremented</td><td>$PC \leftarrow [PC] + 1$</td></tr> <tr> <td>3</td><td>It should be the contents of the address in the MAR</td><td>$MDR \leftarrow [[MAR]]$</td></tr> </table>	Line number	Description of the error	Correct statement	2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$	3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$							
Line number	Description of the error	Correct statement															
2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$															
3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$															
6(b)	1 mark for each correct row	4															
	<table> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr> <tr> <td>11111111</td><td>OR 101</td><td>11111111</td></tr> <tr> <td>00000000</td><td>XOR #15</td><td>00001111</td></tr> <tr> <td>10101010</td><td>LSR #2</td><td>00101010</td></tr> <tr> <td>01010101</td><td>AND 104</td><td>00000000</td></tr> </table>	Current contents of the ACC	Instruction	New contents of the ACC	11111111	OR 101	11111111	00000000	XOR #15	00001111	10101010	LSR #2	00101010	01010101	AND 104	00000000	
Current contents of the ACC	Instruction	New contents of the ACC															
11111111	OR 101	11111111															
00000000	XOR #15	00001111															
10101010	LSR #2	00101010															
01010101	AND 104	00000000															

42. O/N 21/P12/Q7/a

7(a)	1 mark per pair of rows (shaded & unshaded)			3
Event		Hardware Interrupt	Software Interrupt	
Buffer full			✓	} 1
Printer is out of paper		✓		
User has pressed a key on the keyboard		✓		} 1
Division by zero			✓	
Power failure		✓		} 1
Stack overflow			✓	

43. O/N 21/P12/Q8/a,b

8(a)(i)	1 mark for each bullet point to max 2 for each register MAR • Stores the next <u>address</u> to be fetched • ... held in the Program Counter (PC) • The data at this address is then fetched MDR • Stores the data from the address pointed to by the MAR • The data in it is copied to the Current Instruction Register (CIR)	4															
8(a)(ii)	1 mark for a correct register e.g. Program Counter (PC) Current Instruction Register (CIR) Status register Interrupt register	1															
8(b)(i)	1 mark for each correct answer <table border="1"><thead><tr><th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr></thead><tbody><tr><td>01010101</td><td>XOR 101</td><td>1010 0101</td></tr><tr><td>11110000</td><td>AND 104</td><td>1111 0000</td></tr><tr><td>00001111</td><td>LSL #4</td><td>1111 0000</td></tr><tr><td>11111111</td><td>OR 102</td><td>1111 1111</td></tr></tbody></table>	Current contents of the ACC	Instruction	New contents of the ACC	01010101	XOR 101	1010 0101	11110000	AND 104	1111 0000	00001111	LSL #4	1111 0000	11111111	OR 102	1111 1111	4
Current contents of the ACC	Instruction	New contents of the ACC															
01010101	XOR 101	1010 0101															
11110000	AND 104	1111 0000															
00001111	LSL #4	1111 0000															
11111111	OR 102	1111 1111															

8(b)(ii)	1 mark for each correct instruction <table> <tr> <th>Instruction Group</th> <th>Instruction</th> </tr> <tr> <td>Data movement</td> <td>LDM #2</td> </tr> <tr> <td>Input and output of data</td> <td>IN / OUT</td> </tr> <tr> <td>Arithmetic Operations</td> <td>INC ACC / INC IX</td> </tr> <tr> <td>Unconditional and conditional instructions</td> <td>JPN 100 / JMP 100</td> </tr> <tr> <td>Compare instructions</td> <td>CMP 100</td> </tr> </table>	Instruction Group	Instruction	Data movement	LDM #2	Input and output of data	IN / OUT	Arithmetic Operations	INC ACC / INC IX	Unconditional and conditional instructions	JPN 100 / JMP 100	Compare instructions	CMP 100	4
Instruction Group	Instruction													
Data movement	LDM #2													
Input and output of data	IN / OUT													
Arithmetic Operations	INC ACC / INC IX													
Unconditional and conditional instructions	JPN 100 / JMP 100													
Compare instructions	CMP 100													
8(b)(iii)	1 mark for name, 1 mark for description - Indirect addressing - the address to be used is at the given address - Relative addressing - the address to be used is an offset number of locations away, relative to the address of the current instruction - Indexed addressing - form the address from the given address plus the contents of the index register	2												

44. O/N 21/P13/Q6/a,b

6(a)	1 mark for identification of line and description of error 1 mark for the correct statement	4															
	<table> <tr> <th>Line number</th><th>Description of the error</th><th>Correct statement</th></tr> <tr> <td>2</td><td>Program Counter should be incremented, not decremented</td><td>$PC \leftarrow [PC] + 1$</td></tr> <tr> <td>3</td><td>It should be the contents of the address in the MAR</td><td>$MDR \leftarrow [[MAR]]$</td></tr> </table>	Line number	Description of the error	Correct statement	2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$	3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$							
Line number	Description of the error	Correct statement															
2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$															
3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$															
6(b)	1 mark for each correct row	4															
	<table> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr> <tr> <td>11111111</td><td>OR 101</td><td>11111111</td></tr> <tr> <td>00000000</td><td>XOR #15</td><td>00001111</td></tr> <tr> <td>10101010</td><td>LSR #2</td><td>00101010</td></tr> <tr> <td>01010101</td><td>AND 104</td><td>00000000</td></tr> </table>	Current contents of the ACC	Instruction	New contents of the ACC	11111111	OR 101	11111111	00000000	XOR #15	00001111	10101010	LSR #2	00101010	01010101	AND 104	00000000	
Current contents of the ACC	Instruction	New contents of the ACC															
11111111	OR 101	11111111															
00000000	XOR #15	00001111															
10101010	LSR #2	00101010															
01010101	AND 104	00000000															

45. 9608 O/N 21/P11/Q6/a,b

6(a)	One mark for each term in bold. There are three buses that transfer data between components in a computer system. The width of the address bus determines the number of directly accessible memory locations. The control unit sends signals on the control bus to direct the operation of system components. Clock pulses are used to synchronise the components on the motherboard.	5
6(b)	One mark per bullet point to max 5 • The address in the program counter is the address of next item to be fetched • The address is copied into MAR ... • ...using the address bus • The instruction from that address moved/copied from main memory to MDR ... • ...using the data bus • The instruction is transferred from MDR to CIR • The processor's instruction set is used to decode the instruction// the instruction is decoded in the CIR • ...into op code and operand • The processor executes the instruction // the processor processes the data as required • The address in PC is incremented ready for next loop	5

46. 9608 O/N 21/P11/Q8

8

One mark for each set of rows (shaded and unshaded)

4

Instruction address	ACC	Memory address				
		150	151	152	153	160
		13	23	11	0	150
130	13					
131	13					
132					13	
133	150					
134	151					
135						151
136						
130	23					
131	36					
132					36	
133	151					
134	152					
135						152
136						

47. M/J 21/P11/Q3/a,b,c

3(a)	1 mark for each completed statement The Program Counter holds the address of the next instruction to be loaded. This address is sent to the Memory Address Register. The Memory Data Register holds the data fetched from this address. This data is sent to the Current Instruction Register and the Control Unit decodes the instruction's opcode. The Program Counter is incremented.	5
------	--	---

3(b)	1 mark for each shaded set of values <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th><th rowspan="2">IX</th><th rowspan="2">Output</th></tr><tr><th>365</th><th>366</th><th>367</th><th>368</th></tr><tr><td></td><td></td><td>1</td><td>3</td><td>65</td><td>66</td><td>0</td><td></td></tr><tr><td>200</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>2</td><td></td></tr><tr><td>206</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>A</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>3</td><td></td></tr><tr><td>206</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>B</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>209</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				IX	Output	365	366	367	368			1	3	65	66	0		200	1							201								202								203	2							204		2						205						2		206	65							207							A	208								200	2							201								202								203	3							204		3						205						3		206	66							207							B	208								200	3							201								202								209								6
Instruction address	ACC			Memory address						IX	Output																																																																																																																																																																																											
		365	366	367	368																																																																																																																																																																																																	
		1	3	65	66	0																																																																																																																																																																																																
200	1																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	2																																																																																																																																																																																																					
204		2																																																																																																																																																																																																				
205						2																																																																																																																																																																																																
206	65																																																																																																																																																																																																					
207							A																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	2																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	3																																																																																																																																																																																																					
204		3																																																																																																																																																																																																				
205						3																																																																																																																																																																																																
206	66																																																																																																																																																																																																					
207							B																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	3																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
209																																																																																																																																																																																																						
3(c)(i)	<table><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	1	0	1	0	0	1																																																																																																																																																																																												
1	1	0	1	0	1	0	0																																																																																																																																																																																															
3(c)(ii)	1 mark for correct answer The number is divided by 8 (and only whole number retained)	1																																																																																																																																																																																																				

48. M/J 21/P12/Q4/a,b(ii,iii)

4(a)	1 mark for each shaded section / bullet point - Load 65 into ACC - Load 100 into ACC, increment and store in 102 - Load 68 into ACC - Load 101 into ACC, decrement and store in 102 <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="3">Memory address</th></tr><tr><th>100</th><th>101</th><th>102</th></tr><tr><td></td><td></td><td>68</td><td>65</td><td>100</td></tr><tr><td>70</td><td>65</td><td></td><td></td><td></td></tr><tr><td>71</td><td></td><td></td><td></td><td></td></tr><tr><td>72</td><td></td><td></td><td></td><td></td></tr><tr><td>73</td><td></td><td></td><td></td><td></td></tr><tr><td>74</td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td>100</td><td></td><td></td><td></td></tr><tr><td>77</td><td>101</td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td>101</td></tr><tr><td>79</td><td></td><td></td><td></td><td></td></tr><tr><td>70</td><td>68</td><td></td><td></td><td></td></tr><tr><td>71</td><td></td><td></td><td></td><td></td></tr><tr><td>72</td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td>101</td><td></td><td></td><td></td></tr><tr><td>81</td><td>100</td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td>100</td></tr><tr><td>83</td><td></td><td></td><td></td><td></td></tr><tr><td>(70)</td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address			100	101	102			68	65	100	70	65				71					72					73					74					76	100				77	101				78				101	79					70	68				71					72					80	101				81	100				82				100	83					(70)					4
Instruction address	ACC			Memory address																																																																																																
		100	101	102																																																																																																
		68	65	100																																																																																																
70	65																																																																																																			
71																																																																																																				
72																																																																																																				
73																																																																																																				
74																																																																																																				
76	100																																																																																																			
77	101																																																																																																			
78				101																																																																																																
79																																																																																																				
70	68																																																																																																			
71																																																																																																				
72																																																																																																				
80	101																																																																																																			
81	100																																																																																																			
82				100																																																																																																
83																																																																																																				
(70)																																																																																																				
4(b)(ii)	AND	1																																																																																																		
4(b)(iii)	1 mark for AND, 1 mark for #15 AND #15	2																																																																																																		

49. M/J 21/P12/Q5/a,b

5(a)	1 mark for each term correctly inserted The control unit/bus transmits the signals to coordinate events based on the pulses of the (system) clock. The data bus carries data to components, while the address bus carries the address where data is being written to or read from. The arithmetic logic unit/ALU performs mathematical operations and logical comparisons.	5
5(b)	1 mark per bullet point to max 3 per factor. max 4 overall. Number of cores: • Each core processes one <u>instruction</u> per clock pulse • More/multiple cores mean that sequences of instructions can be split between them • ... and so more than one <u>instruction</u> is executed per clock pulse // more sequences of instructions can be run at the same time • More cores decreases the time taken to complete task Clock speed: • Each <u>instruction</u> is executed on a clock pulse // one F-E cycle is run on each clock pulse • ... so the clock speed dictates the number of <u>instructions</u> that can be run per second • The faster the clock speed the more <u>instructions</u> can be run per second	4

50. M/J 21/P13/Q3/a,b,c

3(a)	1 mark for each completed statement The Program Counter holds the address of the next instruction to be loaded. This address is sent to the Memory Address Register. The Memory Data Register holds the data fetched from this address. This data is sent to the Current Instruction Register and the Control Unit decodes the instruction's opcode. The Program Counter is incremented.	5
------	--	----------

3(b)	1 mark for each shaded set of values <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th><th rowspan="2">IX</th><th rowspan="2">Output</th></tr><tr><th>365</th><th>366</th><th>367</th><th>368</th></tr><tr><td></td><td></td><td>1</td><td>3</td><td>65</td><td>66</td><td>0</td><td></td></tr><tr><td>200</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>2</td><td></td></tr><tr><td>206</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>A</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>3</td><td></td></tr><tr><td>206</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>B</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>209</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				IX	Output	365	366	367	368			1	3	65	66	0		200	1							201								202								203	2							204		2						205						2		206	65							207							A	208								200	2							201								202								203	3							204		3						205						3		206	66							207							B	208								200	3							201								202								209								6
Instruction address	ACC			Memory address						IX	Output																																																																																																																																																																																											
		365	366	367	368																																																																																																																																																																																																	
		1	3	65	66	0																																																																																																																																																																																																
200	1																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	2																																																																																																																																																																																																					
204		2																																																																																																																																																																																																				
205						2																																																																																																																																																																																																
206	65																																																																																																																																																																																																					
207							A																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	2																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	3																																																																																																																																																																																																					
204		3																																																																																																																																																																																																				
205						3																																																																																																																																																																																																
206	66																																																																																																																																																																																																					
207							B																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	3																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
209																																																																																																																																																																																																						
3(c)(i)	<table><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	1	0	1	0	0	1																																																																																																																																																																																												
1	1	0	1	0	1	0	0																																																																																																																																																																																															
3(c)(ii)	1 mark for correct answer The number is divided by 8 (and only whole number retained)	1																																																																																																																																																																																																				

51. 9608 M/J 21/P11/Q3/a,b,d

3(a)	1 mark for each set of shaded rows <table><tr><th rowspan="2">Instruction Address</th><th rowspan="2">ACC</th><th colspan="3">Memory Address</th></tr><tr><th>200</th><th>201</th><th>202</th></tr><tr><td></td><td></td><td>2</td><td>0</td><td>200</td></tr><tr><td>100</td><td>2</td><td></td><td></td><td></td></tr><tr><td>101</td><td></td><td></td><td></td><td></td></tr><tr><td>102</td><td></td><td></td><td></td><td></td></tr><tr><td>103</td><td>1</td><td></td><td></td><td></td></tr><tr><td>104</td><td></td><td>1</td><td></td><td></td></tr><tr><td>105</td><td></td><td></td><td></td><td></td></tr><tr><td>101</td><td></td><td></td><td></td><td></td></tr><tr><td>102</td><td></td><td></td><td></td><td></td></tr><tr><td>103</td><td>0</td><td></td><td></td><td></td></tr><tr><td>104</td><td></td><td>0</td><td></td><td></td></tr><tr><td>105</td><td></td><td></td><td></td><td></td></tr><tr><td>101</td><td></td><td></td><td></td><td></td></tr><tr><td>102</td><td></td><td></td><td></td><td></td></tr><tr><td>106</td><td></td><td></td><td></td><td></td></tr></table>	Instruction Address	ACC	Memory Address			200	201	202			2	0	200	100	2				101					102					103	1				104		1			105					101					102					103	0				104		0			105					101					102					106					3
Instruction Address	ACC			Memory Address																																																																																	
		200	201	202																																																																																	
		2	0	200																																																																																	
100	2																																																																																				
101																																																																																					
102																																																																																					
103	1																																																																																				
104		1																																																																																			
105																																																																																					
101																																																																																					
102																																																																																					
103	0																																																																																				
104		0																																																																																			
105																																																																																					
101																																																																																					
102																																																																																					
106																																																																																					
3(b)	LDI 202	1																																																																																			
3(d)	1 mark per bullet point to max 3 e.g. - Zero - Carry - Overflow - Sign/negative - Compare results - Parity	3																																																																																			

52. 9608 M/J 21/P12/Q2

2	1 mark for each register correctly described Register Notation MDR CIR MAR PC Description Holds the op code and operand of an instruction ready for it to be decoded Holds the address of the next instruction to be read Holds flags that are set when the Arithmetic and Logic Unit (ALU) executes instructions Holds data read from, or to be written to, memory Holds the current value in the Index Register Holds the address where data is to be written to or read from Holds the result of the last instruction executed by the ALU	4
---	---	---

53. 9608 M/J 21/P12/Q3/a,b,d

3(a)	1 Mark for each set of shaded rows						5																																																																																																																																																																										
<table><tr><th rowspan="2">Instruction Address</th><th rowspan="2">ACC</th><th colspan="4">Memory Address</th></tr><tr><th>100</th><th>101</th><th>102</th><th>103</th></tr><tr><td></td><td></td><td>1</td><td>3</td><td>0</td><td>100</td><td></td></tr><tr><td>50</td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>51</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>52</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>53</td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>54</td><td></td><td></td><td></td><td>1</td><td></td><td></td></tr><tr><td>55</td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>56</td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>57</td><td></td><td>2</td><td></td><td></td><td></td><td></td></tr><tr><td>58</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>51</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>52</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>53</td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>54</td><td></td><td></td><td></td><td>3</td><td></td><td></td></tr><tr><td>55</td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>56</td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>57</td><td></td><td>3</td><td></td><td></td><td></td><td></td></tr><tr><td>58</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>51</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>52</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>59</td><td>6</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>60</td><td></td><td></td><td></td><td>6</td><td></td><td></td></tr><tr><td>61</td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>							Instruction Address	ACC	Memory Address				100	101	102	103			1	3	0	100		50	1						51							52							53	1						54				1			55	1						56	2						57		2					58							51							52							53	3						54				3			55	2						56	3						57		3					58							51							52							59	6						60				6			61						
Instruction Address	ACC	Memory Address																																																																																																																																																																															
		100	101	102	103																																																																																																																																																																												
		1	3	0	100																																																																																																																																																																												
50	1																																																																																																																																																																																
51																																																																																																																																																																																	
52																																																																																																																																																																																	
53	1																																																																																																																																																																																
54				1																																																																																																																																																																													
55	1																																																																																																																																																																																
56	2																																																																																																																																																																																
57		2																																																																																																																																																																															
58																																																																																																																																																																																	
51																																																																																																																																																																																	
52																																																																																																																																																																																	
53	3																																																																																																																																																																																
54				3																																																																																																																																																																													
55	2																																																																																																																																																																																
56	3																																																																																																																																																																																
57		3																																																																																																																																																																															
58																																																																																																																																																																																	
51																																																																																																																																																																																	
52																																																																																																																																																																																	
59	6																																																																																																																																																																																
60				6																																																																																																																																																																													
61																																																																																																																																																																																	
3(b)	LDD 100						1																																																																																																																																																																										

3(b)	1 mark per bullet point max 2 marks • Direct • Indirect • Relative	2
3(d)	1 mark for each correctly completed item MAR ← [PC] PC ← [PC] + 1 MDR ← [[MAR]] CIR ← [MDR]	3

55. M/J 20/P11/Q6/a,b

6(a)(i)	1 mark for each correct answer A: The number 193 B: The data in memory location 193 C: The data in the memory location found by adding the contents of the IX to 193	3
6(a)(ii)	1 mark each correct answer • Indirect • Relative	2
6(b)	1 mark for correctly naming register, 1 mark for appropriate role • Program counter // PC • Stores the address of the next instruction to be fetched • Memory address register // MAR • Stores the address where data/instruction is to be read from or saved to • Memory data register // MDR • Stores data that is about to be written to memory // Stores data that has just been read from memory • Current instruction register // CIR • Stores the instruction that is currently being decoded/executed	4

56. M/J 20/P12/Q2/c(iii)

2(c)(iii)	1 mark for benefit, 1 mark for expansion for max 2 benefits • Fast data transfer • ... useful when transferring large files such as video files • Automatic connection / plug-and-play • ... so usually there is no need to install separate device drivers • USB is a standard adopted by many manufacturers • ... meaning all new computers will be equipped with USB ports • Devices may be powered or charged through USB • ... so external devices may be charged while working at the computer	4
-----------	---	----------

57. M/J 20/P12/Q5/a,c

5(a)	1 mark for one letter in the correct place, 2 marks for all three correct 2 B 4 A 5 C	2
------	---	----------

5(c)

1 mark for each highlighted section block

6

Instruction address	ACC	Memory address			
		100	101	102	103
		1	2	3	0
20	0				
21					
22					
23	1				
24	3				
25		3			
26	0				
27	1				
28					1
29					
20	1				
21					
22					
23	3				
24	5				
25		5			
26	1				
27	2				
28					2
29					
20	2				
21					
22					
30					

58. M/J 20/P13/Q2/c

2(c)(i)	1 mark - To provide the connection to peripheral devices - To provide an interface between the computer and other devices	1
2(c)(ii)	1 mark - USB - HDMI - SCSI - Firewire - Ethernet - Any other acceptable port	1

59. O/N 19/P11/Q1/a,b,c

1(a)	1 mark for each correct line Register Current Instruction Register Memory Address Register Program Counter Memory Data Register Description Stores the data that has just been read from memory, or is about to be written to memory Stores the instruction that is being decoded and executed Stores the address of the input device from which the processor accesses the instruction Stores the address of the next instruction to be read Stores the address of the memory location about to be written to or read from	4
1(b)(i)	1 mark for naming, 1 mark for purpose for each bus - Data bus - Carries data between the processor and memory / carries data that is currently being processed. - Control bus - Transmits signals between the control unit and the other components	4
1(b)(ii)	Significant increase in the number of directly addressed memory locations // increases the number of directly addressable memory locations from 2^{16} to 2^{32}	1

1(c)	1 mark for each correctly inserted term A macro is a sequence of instructions that are given an identifier. These instructions may need to be executed several times. A directive is an instruction that tells the assembler to do something. It is not a program instruction. The processor's instruction set can be put into several groups. One of these groups is data movement // input and output // arithmetic operations // jump instructions // compare instructions // modes of addressing.	3
------	---	----------

60. O/N 19/P13/Q7

7	1 mark for each correct addressing mode	5												
<table><tr><th>Addressing mode</th><th>Description</th></tr><tr><td>Relative</td><td>Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).</td></tr><tr><td>Indirect</td><td>Load the contents of the address held at the given address to ACC.</td></tr><tr><td>Direct</td><td>Load the contents of the given address to ACC.</td></tr><tr><td>Indexed</td><td>Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.</td></tr><tr><td>Immediate</td><td>Load the given value directly to ACC.</td></tr></table>			Addressing mode	Description	Relative	Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).	Indirect	Load the contents of the address held at the given address to ACC.	Direct	Load the contents of the given address to ACC.	Indexed	Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.	Immediate	Load the given value directly to ACC.
Addressing mode	Description													
Relative	Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).													
Indirect	Load the contents of the address held at the given address to ACC.													
Direct	Load the contents of the given address to ACC.													
Indexed	Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.													
Immediate	Load the given value directly to ACC.													

61. M/J 19/P11/Q5/a,b,c

5(a)	1 mark for two correct ticks, 2 marks for three correct ticks <table border="1"> <thead> <tr> <th>Task</th><th>First pass</th><th>Second pass</th></tr> </thead> <tbody> <tr> <td>Creation of symbol table</td><td>✓</td><td></td></tr> <tr> <td>Expansion of macros</td><td>✓</td><td></td></tr> <tr> <td>Generation of object code</td><td></td><td>✓</td></tr> <tr> <td>Removal of comments</td><td>✓</td><td></td></tr> </tbody> </table>	Task	First pass	Second pass	Creation of symbol table	✓		Expansion of macros	✓		Generation of object code		✓	Removal of comments	✓		2
Task	First pass	Second pass															
Creation of symbol table	✓																
Expansion of macros	✓																
Generation of object code		✓															
Removal of comments	✓																
5(b)	1 mark per bullet point to max 2 • Data movement • Input and output of data • Arithmetic operations • Jump instructions • Compare instructions	2															

5(c)	1 mark per bullet point - Storing 0 in 300 (line 21) - Loading 65 (line 28) - Outputting A (line 29) - Loading 0 (line 30), incrementing ACC (line 31) and storing in 300 (line 32) - Incrementing IX (line 33) - Loading 67 (line 24) and adding 33 (line (25)) - Outputting d (line 26) - Loading 1 (line 30), incrementing ACC (line 31), storing in 300 (line 32) and incrementing IX (line 33)	8
------	---	----------

62. M/J 19/P12/Q3/a,b,c

3(a)	1 mark for each error and correction - Line 02 should be +1 not -1 // $PC \leftarrow [PC] + 1$ - Line 03 should be double brackets around MAR // $MDR \leftarrow [[MAR]]$ - Line 04 should be MDR not MAR // $CIR \leftarrow [MDR]$	3
3(b)	1 mark for each group to max. 2 - Data movement - Arithmetic operations (Unconditional and conditional) jump instructions - Compare instructions - Modes of addressing	2
3(c)	1 mark per bullet - Storing 0 in 401 (line 51) - Loading memory location 300, value 2 to ACC (line 52) - Adding 64 to ACC to give 66 (line 55) - Outputting B (line 56) - Load 0 (line 57), increment ACC (line 58) and store 1 in 401 (line 59) - Incrementing IX (line 60) - Loading 5 (line 52), adding 64 (line 55), outputting E (line 56) loading 1 (line 57), incrementing ACC (line 58), storing 2 in 401 (line 59) and incrementing IX (line 60) - Load 0 (line 52) <u>and end</u>	8

63. M/J 19/P13/Q4/a,b,c

4(a)	1 mark per bullet - LDM #300 The (denary) number 300 is loaded (into the register) - LDD 300 The <u>contents</u> of address 300 are loaded (into the register)	2
------	--	----------

4(b)	1 mark for JPN correct 1 mark for both of ADD and DEC correct 1 mark for LDR correct <table><tr><th></th><th>Description</th><th>Jump instruction</th><th>Arithmetic operation</th><th>Data movement</th></tr><tr><td>LDR #3</td><td>Load the number 3 to the Index Register</td><td></td><td></td><td>✓</td></tr><tr><td>ADD #2</td><td>Add 2 to the Accumulator</td><td></td><td>✓</td><td></td></tr><tr><td>JPN 22</td><td>Move to the instruction at address 22</td><td>✓</td><td></td><td></td></tr><tr><td>DEC ACC</td><td>Subtract 1 from the Accumulator</td><td></td><td>✓</td><td></td></tr></table>		Description	Jump instruction	Arithmetic operation	Data movement	LDR #3	Load the number 3 to the Index Register			✓	ADD #2	Add 2 to the Accumulator		✓		JPN 22	Move to the instruction at address 22	✓			DEC ACC	Subtract 1 from the Accumulator		✓		3
	Description	Jump instruction	Arithmetic operation	Data movement																							
LDR #3	Load the number 3 to the Index Register			✓																							
ADD #2	Add 2 to the Accumulator		✓																								
JPN 22	Move to the instruction at address 22	✓																									
DEC ACC	Subtract 1 from the Accumulator		✓																								
4(c)(i)	1 mark for hardware interrupt 1 mark for software interrupt For example: Hardware interrupt • Printer out of paper • No CD in drive Software interrupt • A running program needs input • Runtime error, e.g. division by zero	2																									
4(c)(ii)	1 mark per bullet to max 5 • At the start / end of each fetch-execute cycle the processor checks for interrupt(s) • Check if an interrupt flag is set // Check if bit set in interrupt register • Processor identifies source of interrupt • Processor checks priority of interrupt • If interrupt priority is high enough // Lower priority interrupts are disabled • Processor saves current contents of registers // saves current job on stack • Processor calls interrupt handler / Interrupt Service Routine (ISR) • Address of ISR is loaded into Program Counter (PC) • When servicing of interrupt complete, processor restores registers // job from stack is restored • Lower priority interrupts are re-enabled • Processor continues with next F–E cycle	5																									

64. O/N 18/P11/Q4/a,b,d

4(a)(i)	1 mark for each type of addressing Direct addressing - The operand is the address where the data is stored Indirect addressing - The operand is an address, that address holds another address where the data is stored	2												
4(a)(ii)	1 mark per bullet point Direct addressing: 20 is the address of the data Indirect addressing: 20 is an address which holds the address where the data is stored	2												
4(b)	1 mark for 1 correct tick 2 marks for 3 correct ticks <table border="1"> <thead> <tr> <th>Instruction</th><th>Symbolic</th><th>Absolute</th></tr> </thead> <tbody> <tr> <td>ADD 90</td><td></td><td>✓</td></tr> <tr> <td>CMP found</td><td>✓</td><td></td></tr> <tr> <td>STO 20</td><td></td><td>✓</td></tr> </tbody> </table>	Instruction	Symbolic	Absolute	ADD 90		✓	CMP found	✓		STO 20		✓	2
Instruction	Symbolic	Absolute												
ADD 90		✓												
CMP found	✓													
STO 20		✓												
4(d)	1 mark per bullet point - Outputting * (instruction 71) - Storing 130 in 203 (instruction 72) - Loading, incrementing accumulator and storing in 204 (instructions 73, 74 and 75) - Incrementing Index Register (instruction 76) - Loading 133, comparing and jumping to 81 (instructions 77, 78 and 79) - Loading, comparing and jumping to 74 (instructions 81, 82 and 83) - Incrementing accumulator, storing in 204 and incrementing index register, loading 130 (instructions 74–79) - Outputting * to end (instructions 80–84)	8												

65. O/N 18/P12/Q3/a,b,d

3(a)(i)	1 mark per bullet point Absolute addressing: - The operand is a numeric address // The numeric address is given // referring directly to a memory location Symbolic addressing: - The operand is a word/symbol // A word/symbol represents the memory location/address	2
---------	--	----------

3(a)(ii)	1 mark per example Absolute addressing: For example, <code>ADD 230</code> Symbolic addressing: For example, <code>ADD num1</code>	2
3(b)(i)	1 mark per bullet point Indexed addressing: - The address to be used is formed by: operand + the contents of the Index Register (IX) Immediate addressing: - The operand is not an address // the operand is the actual value to be loaded	2
3(b)(ii)	1 mark per example Indexed: For example, <code>LDX 20</code> Immediate: For example, <code>ADD #20</code>	2
3(d)	1 mark per bullet point - Loading 2, comparing with 104 (instructions 40 and 41) - Loading 302 (instruction 43) - Comparing, and branching to 47 (instructions 44, 45) - Loading, decrementing accumulator and storing (instructions 47, 48 and 49) - Incrementing Index Register (instruction 50) - Loading 303, comparing and outputting + (instructions 43–46) - Loading, decrementing accumulator and storing, incrementing Index Register and end (instructions 47–51, 41, 42 and 54)	7

[illegible]

66. O/N 18/P12/Q4/a,b,c

4(a)	1 mark per correct line, max 3 <table><tr><th>Line number of error</th><th>Correct notation</th></tr><tr><td>1</td><td>MAR ← [PC]</td></tr><tr><td>3</td><td>MDR ← [[MAR]]</td></tr><tr><td>4</td><td>CIR ← [MDR]</td></tr><tr><td>2</td><td>PC ← [PC] + 1</td></tr></table>	Line number of error	Correct notation	1	MAR ← [PC]	3	MDR ← [[MAR]]	4	CIR ← [MDR]	2	PC ← [PC] + 1	3
Line number of error	Correct notation											
1	MAR ← [PC]											
3	MDR ← [[MAR]]											
4	CIR ← [MDR]											
2	PC ← [PC] + 1											
4(b)(i)	1 mark for each event to max 3 For example: • Hardware fault // Example of hardware fault • I/O request // Example of I/O request • Program/software error // Example of software error • End of a time-slice	3										
4(b)(ii)	1 mark per bullet point to max 5 • At the <u>end</u> of each fetch–execute cycle the processor checks for interrupt(s) • Check if an interrupt flag is set // Check if bit set in interrupt register • Processor identifies source of interrupt • Processor checks priority of interrupt • If interrupt priority is high enough // Lower priority interrupts are disabled • Processor saves current contents of registers • Processor calls interrupt handler / Interrupt Service Routine (ISR) • Address of ISR is loaded into Program Counter (PC) • When servicing of interrupt complete, processor restores registers • Lower priority interrupts are re-enabled • Processor continues with next F–E cycle	5										

4(c)	1 mark for 1 correct connection 2 marks for all 3 correct connections <table><tr><th>Bus</th><th>Description</th></tr><tr><td>Control Bus</td><td>Uni-directional (one direction)</td></tr><tr><td>Address Bus</td><td>Bi-directional (two directions)</td></tr><tr><td>Data Bus</td><td>Bi-directional (two directions)</td></tr></table> <pre>graph LR; CB[Control Bus] --- BD[Bi-directional (two directions)]; AB[Address Bus] --- UD[Uni-directional (one direction)]; DB[Data Bus] --- UD;</pre>	Bus	Description	Control Bus	Uni-directional (one direction)	Address Bus	Bi-directional (two directions)	Data Bus	Bi-directional (two directions)	2
Bus	Description									
Control Bus	Uni-directional (one direction)									
Address Bus	Bi-directional (two directions)									
Data Bus	Bi-directional (two directions)									

67. O/N 18/P13/Q2/a,b(iv),c

2(a)	1 mark per bullet point Relative addressing: • A number (the offset) is added to the base address to give the actual address Indexed addressing: • The contents of the index register are added to operand to give the actual address	2								
2(b)(iv)	<table border="1"><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	0	1	1	1
1	1	1	1	0	0	1	1			
2(c)	1 mark per bullet point • Loading 8 (instruction 20) • Comparison and loading 453 (instructions 21–23) • Outputting & (instruction 26) • Loading, incrementing and storing in 96 (instructions 27–29) • Incrementing Index Register (instruction 30) • Jumping and loading 452 (instruction 23) • Jumping, loading, incrementing, storing in 96, incrementing IX and end (instructions 24–32)	7								

68. O/N 18/P13/Q6/a,b

6(a)(i)	$MAR \leftarrow [PC]$ $PC \leftarrow PC + 1$ $MDR \leftarrow [MAR]$ $CIR \leftarrow [MDR]$	3
6(a)(ii)	1 mark from: - The contents of the MAR is an address, it is the contents of that address which is transferred to MDR - The contents of the address pointed to by the MAR is transferred to the MDR	1
6(b)	1 mark per bullet point to max 2 - A signal from a source / device - Telling the processor its attention is needed	2

69. M/J 18/P11/Q8/a,b

8(a)	1 mark per bullet to max 2 - Small piece / word of (fast) memory - Part of the processor - Temporary storage of data - Data is about to be / has been processed	2
8(b)(i)	1 mark per bullet to max 2 - Stores / holds data / instruction when fetched from memory - Stores / holds data which is being written to memory - The location accessed is the address held in the Memory Address Register (MAR)	2
8(b)(ii)	1 mark per bullet to max 2 - Current Instruction Register (CIR) - Memory Address Register (MAR) - Program Counter (PC) - Accumulator (ACC) - Index Register (IX) - Status Register - Interrupt Register	2

70. M/J 18/P13/Q3/a,b

3(a)	1 mark per bullet to max 2 for each group ALU performs arithmetic operations - And logical operations / comparisons Control Unit sends / receives signals - Synchronises operations - to control operations // execution of instructions - Accept by example e.g. Input output // flow of data	4
------	---	----------

3(b)	1 mark per bullet to max 2 for each group Status Register is interpreted as independent bits / flags - Each flag is set depending on an event - An example: addition overflow / result of operation is zero etc. Program Counter stores the <u>address</u> of the <u>next</u> instruction to be fetched	4
------	--	----------

71. O/N 17/P12/Q4/a,b,d,e

4(a)	1 Mark for each correct answer A – General purpose registers B – System clock C – ALU E – Control bus F – Address bus	5																																							
4(b)	1 Mark per bullet, max 2 - The clock sends out a number of pulses in a given time interval (clock speed) - Each processor instruction takes a certain number of clock cycles to execute - The higher the clock frequency, the shorter the execution time for the instruction // Increasing the clock frequency improves performance	2																																							
4(d)	Mark as shown <table border="1"> <thead> <tr> <th>ACC</th><th>Offset</th><th>OUTPUT</th></tr> </thead> <tbody> <tr> <td></td><td>10</td><td></td></tr> <tr> <td>50</td><td></td><td>2</td></tr> <tr> <td>10</td><td></td><td></td></tr> <tr> <td>11</td><td>11</td><td></td></tr> <tr> <td>65</td><td></td><td></td></tr> <tr> <td></td><td></td><td>A</td></tr> <tr> <td>11</td><td></td><td></td></tr> <tr> <td>12</td><td>12</td><td></td></tr> <tr> <td>89</td><td></td><td>Y</td></tr> <tr> <td>12</td><td></td><td></td></tr> <tr> <td>13</td><td>13</td><td></td></tr> <tr> <td>32</td><td></td><td></td></tr> </tbody> </table> 1 Mark for these two values, as first instructions 1 Mark for this value, in any row 1 Mark for this value, in any row 1 Mark for this value, after 65, nothing in between 1 Mark for the rest	ACC	Offset	OUTPUT		10		50		2	10			11	11		65					A	11			12	12		89		Y	12			13	13		32			5
ACC	Offset	OUTPUT																																							
	10																																								
50		2																																							
10																																									
11	11																																								
65																																									
		A																																							
11																																									
12	12																																								
89		Y																																							
12																																									
13	13																																								
32																																									

4(e)

Mark as follows:

6

Table entries:

1 Mark per bullet, max 4

- EndProg
- 2 × Unknown
- 9
- 14
- 8

Numbering:

1 Mark per bullet, max 2

- Relative address of Value is numbered 6
- Number given for EndProg is next number in sequence to relative address of Value
- All numbers correct – award 2 marks

Symbolic address	Relative address
<u>StartProg</u>	0
Offset	UNKNOWN 9 (1)
Value	UNKNOWN (6) 14 (1)
<u>EndProg</u> (7)	UNKNOWN (8) 8 (9)

72. O/N 17/P13/Q4/a,b,c,d

4(a)	A – System clock B – Control unit C – Main memory E – Control bus F – Data bus	5
------	---	---

4(b)	<table><tr><th>ACC</th><th>CountDown</th><th>OUTPUT</th></tr><tr><td></td><td>15</td><td></td></tr><tr><td>67</td><td></td><td>C</td></tr><tr><td>15</td><td></td><td></td></tr><tr><td>14</td><td>14</td><td></td></tr><tr><td>51</td><td></td><td>3</td></tr><tr><td>14</td><td></td><td></td></tr><tr><td>13</td><td>13</td><td></td></tr><tr><td>32</td><td></td><td></td></tr><tr><td>88</td><td></td><td>x</td></tr></table>	ACC	CountDown	OUTPUT		15		67		C	15			14	14		51		3	14			13	13		32			88		x	(1) (1) + (1) (1) (1)	5
	ACC	CountDown	OUTPUT																														
		15																															
	67		C																														
	15																																
	14	14																															
	51		3																														
	14																																
	13	13																															
	32																																
88		x																															
4(c)	Three marks from: - The assembler scans the assembly language instructions in sequence - When it meets a symbolic address checks to see if already in symbol table - If not, it adds it to the symbol table in the symbolic address column - If it is already in symbol table check if absolute address known - If the absolute address is known, it is entered in the appropriate cell - If the absolute address is not known mark / leave as unknown	Max 3																															
4(d)(i)	The op code / mnemonic / instruction table	1																															
4(d)(ii)	A – 1110 0110 0110 1000 (1) (1) B – E6 68 (1)	3																															

73. M/J 17/P11/Q4/a,b,c,d(ii)

4(a)(i)	500	1
4(a)(ii)	496	1
4(a)(iii)	502	1
4(a)(iv)	86	1
4(b)	00000001 00010001 00000110 01100001 Both correct op codes Operand 0001 0001 Operand 0110 0001	3 1 1 1
4(c)	256	1

4(d)(ii)	LDI 63		2
	LDI 63	1 1	

74. M/J 17/P12/Q5/a,b,c(ii)

5(a)(i)	351	1
5(a)(ii)	355	1
5(a)(iii)	22	1
5(a)(iv)	86	1
5(b)	Op code 0 0 0 1 0 0 1 0 0 0 0 1 0 1 0 1 Operand 0 1 0 0 0 0 1 1 0 0 0 0 0 1 1 1 Both correct op codes Operand 0100 0011 Operand 0000 0111 1 1 1	3
5(c)(ii)	LDR #77 LDR #77 1 1	2

75. M/J 17/P13/Q4/a,b,c,d(ii)

4(a)(i)	500	1
4(a)(ii)	496	1
4(a)(iii)	502	1
4(a)(iv)	86	1
4(b)	00000001 00010001 00000110 01100001 Both correct op codes Operand 0001 0001 Operand 0110 0001 1 1 1	3
4(c)	256	1

4(d)(ii)	LDI 63		2
	LDI 63	1 1	

(ii) **ONE** mark for Accumulator contents, **ONE** mark for the explanation.

1	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

- Index Register holds the value 4; $101 + 4 = 105$ so load data from address 105

[2]

(iii) **ONE** mark for Accumulator contents, **TWO** marks for the explanation.

0	1	0	0	1	0	1	1
---	---	---	---	---	---	---	---

- Memory address 103 contains the value 107
- So address 107 is the address from which to load the data

[3]

(b) **ONE** mark for each correct row.

ACC	Memory address			
	810	811	812	813
	28	41	0	0
28				
29				
			29	
41				
70				
				70

[6]

78. O/N 16/P13/Q8/a,b**8 (a) (i)**

Accumulator:	1	0	0	1	0	1	1	1
--------------	---	---	---	---	---	---	---	---

[1]

(ii) One mark for answer and two marks for explanation

Accumulator:	1	1	0	0	0	0	1	0
--------------	---	---	---	---	---	---	---	---

- Index Register contains $1001 = 9$
- $800 + 9 = 809$

[3]

(b) (i) ONE mark for each correct row.

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	
40					
90			90		
90			90		
					Z

[4]

(ii) 107

[1]

79. M/J 16/P11/Q3/a,b**3 (a) Four points from:**

[4]

- The Program Counter (PC) holds the address of the next instruction to be fetched
- The address in the Program Counter (PC) is copied to the Memory Address Register (MAR)
- The Program Counter (PC) is incremented
- The instruction is copied to the Memory Data Register (MDR)
 - from the address held in the Memory Address Register (MAR)
- The instruction from the Memory Data Register (MDR) is copied to the Current Instruction Register (CIR)

(b) One mark for each statement or letter in the correct place.

[4]

At the end of the cycle for the current instruction **B**

If the interrupt flag is set, **D, A** and **C**

The interrupted program continues its execution

At the end of the cycle for the current instruction the processor checks if there is an interrupt.

If the interrupt flag is set, the register contents are saved, the address of the Interrupt Service Routine (ISR) is loaded to the Program Counter (PC) and when the ISR completes, the processor restores the register contents.

The interrupted program continues its execution.

80. M/J 16/P11/Q9/a,b

9 (a) (i) One mark for the contents of the accumulator and **one mark** for the reason.

[2]

Accumulator contents: 0100 0101

Reason:

Address is 60

Contents of the index register is 8

And $60 + 8 = 68$ in denary gives the address

The contents of which is 0100 0101 in binary.

(ii) 0000 0111

[1]

(b)

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50		20						
51		21						
52						21		
53		100						
54		120						
55								
56								
57								
59							2	
60		20						
61		120						
62								'x'
63								

One mark for each shaded block.

[7]

- Contents of the Accumulator in first 2 lines (instruction addresses 50 and 51)
- Updating address 103 (instruction 52)
- Loading the Accumulator and addition (instructions 53 and 54)
- Not executing instruction 58
- Incrementing the index register (instruction 59)
- Loading the Accumulator and addition (instructions 60 and 61)
- Correct output of 'x' (instruction 62)

81. M/J 16/P12/Q3/a,b**3 (a) Four points from:****[4]**

- The Program Counter (PC) holds the address of the next instruction to be fetched
- The address in the Program Counter (PC) is copied to the Memory Address Register (MAR)
- The Program Counter (PC) is incremented
- The instruction is copied to the Memory Data Register (MDR)
 - from the address held in the Memory Address Register (MAR)
- The instruction from the Memory Data Register (MDR) is copied to the Current Instruction Register (CIR)

(b) One mark for each statement or letter in the correct place.**[4]**

At the end of the cycle for the current instruction **B**

If the interrupt flag is set, **D, A** and **C**

The interrupted program continues its execution

At the end of the cycle for the current instruction the processor checks if there is an interrupt.
If the interrupt flag is set, the register contents are saved, the address of the Interrupt Service Routine (ISR) is loaded to the Program Counter (PC) and when the ISR completes, the processor restores the register contents.

The interrupted program continues its execution.

82. M/J 16/P12/Q9/a,b**9 (a) (i) One mark** for the contents of the accumulator and **one mark** for the reason.**[2]**

Accumulator contents: 0100 0101

Reason:

Address is 60

Contents of the index register is 8

And $60 + 8 = 68$ in denary gives the address

The contents of which is 0100 0101 in binary.

(ii) 0000 0111

[1]

(b)

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50		20						
51		21						
52						21		
53		100						
54		120						
55								
56								
57								
59							2	
60		20						
61		120						
62								'x'
63								

One mark for each shaded block.

[7]

- Contents of the Accumulator in first 2 lines (instruction addresses 50 and 51)
- Updating address 103 (instruction 52)
- Loading the Accumulator and addition (instructions 53 and 54)
- Not executing instruction 58
- Incrementing the index register (instruction 59)
- Loading the Accumulator and addition (instructions 60 and 61)
- Correct output of 'x' (instruction 62)

83. M/J 16/P13/Q4/a,b**4 (a)** 11001110**[1]****(b)****[7]**

Instruction	Working space	ACC	Memory address				IX	OUTPUT
			90	91	92	93		
			2	90	55	34	2	
20		55						
21		54						
22			54					
23							3	
24		34						
25		33						
26								
27								
28								
31		67						
32						67		
33								'C'
34								

One mark each for:

- Instruction 20
- Instructions 21 and 22
- Instruction 23
- Instructions 24 and 25
- Not executing instructions 29 and 30
- Instructions 31 and 32
- Correct output

84. O/N 15/P11/Q2/a,b

2 (a) A = control bus

B = address bus

C = data bus

[3]

(b) Program Counter – stores the address of next instruction to be executed

Memory Data Register – stores the data in transit between memory and other registers //
holds the instruction before it is passed to the CIR

Current Instruction Register – stores the current instruction being executed

Memory Address Register – stores the address of the memory location which is about to be
accessed [4]

85. O/N 15/P13/Q2/a,b

2 (a) A = control bus

B = address bus

C = data bus

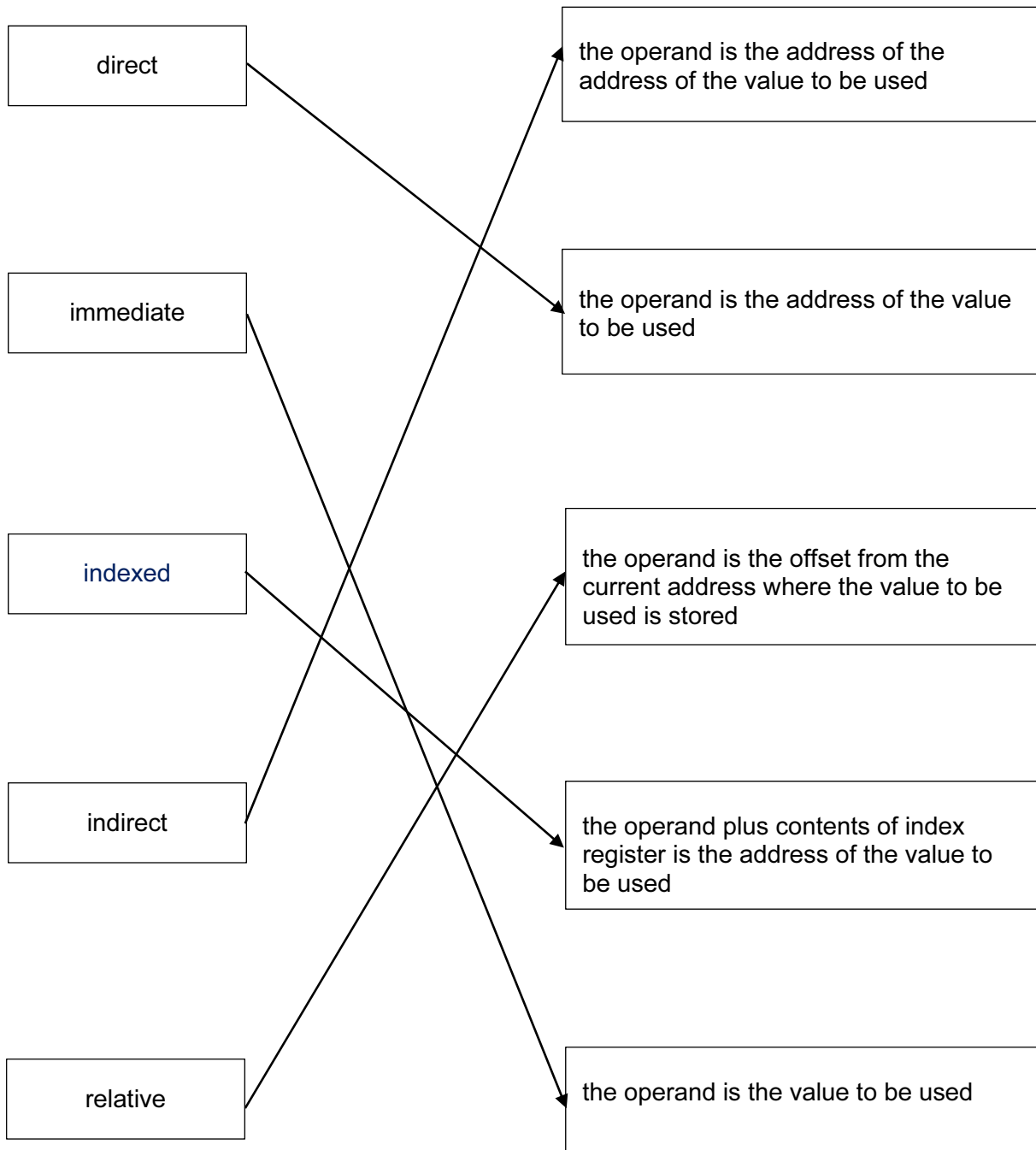
[3]

(b) Program Counter – stores the address of next instruction to be executed

Memory Data Register – stores the data in transit between memory and other registers //
holds the instruction before it is passed to the CIR

Current Instruction Register – stores the current instruction being executed

Memory Address Register – stores the address of the memory location which is about to be
accessed [4]

86. M/J 15/P11/Q3**3****[4]**

87. M/J 15/P11/Q8/a,b,c

8 (a) maximum of 2 marks for data bus width and **maximum of 2 marks** for clock speed

data bus width

- the width of the data bus determines the number of bits that can be simultaneously transferred
- increasing the width of the data bus increases the number of bits/amount of data that can be moved at one time (or equivalent)
- ...hence improving processing speed as fewer transfers are needed
- By example: e.g. double the width of the data bus moves 2x data per clock pulse

clock speed

- determines the number of cycles the CPU can execute per second
 - increasing clock speed increases the number of operations/number of fetch-execute cycles that can be carried out per unit of time
 - ...however, there is a limit on clock speed because the heat generated by higher clock speeds cannot be removed fast enough
- [3]**

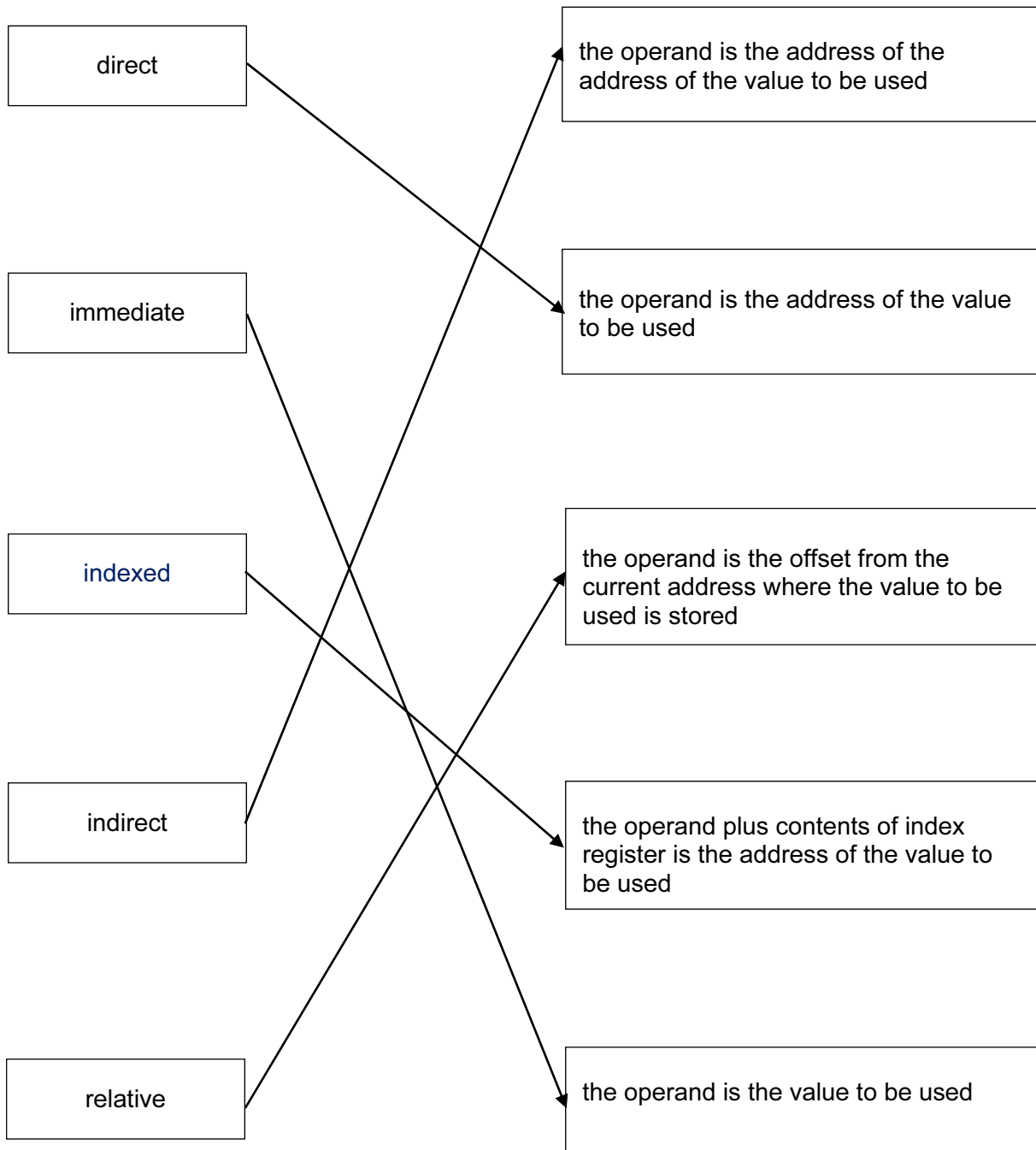
(b) Any two from:

- devices automatically detected and configured when first attached/plug and play
 - it is nearly impossible to wrongly connect a device
 - USB has become an industrial standard
 - supported by many operating systems
 - USB 3.0 allows full duplex data transfer
 - later versions are backwards compatible with earlier USB systems
 - allows power to be drawn to charge portable devices
- [2]**

(c)

Description of stage	Sequence number
the instruction is copied from the Memory Data Register (MDR) and placed in the Current Instruction Register (CIR)	3
the instruction is executed	6
the instruction is decoded	5
the address contained in the Program Counter (PC) is copied to the Memory Address Register (MAR)	1
the value in the Program Counter (PC) is incremented so that it points to the next instruction to be fetched	4
the instruction is copied from the memory location contained in the Memory Address Register (MAR) and is placed in the Memory Data Register (MDR)	2

[6]

88. M/J 15/P12/Q3**3****[4]**

89. M/J 15/P12/Q8/a,b,c

8 (a) maximum of 2 marks for data bus width and **maximum of 2 marks** for clock speed

data bus width

- the width of the data bus determines the number of bits that can be simultaneously transferred
- increasing the width of the data bus increases the number of bits/amount of data that can be moved at one time (or equivalent)
- ...hence improving processing speed as fewer transfers are needed
- By example: e.g. double the width of the data bus moves 2x data per clock pulse

clock speed

- determines the number of cycles the CPU can execute per second
 - increasing clock speed increases the number of operations/number of fetch-execute cycles that can be carried out per unit of time
 - ...however, there is a limit on clock speed because the heat generated by higher clock speeds cannot be removed fast enough
- [3]**

(b) Any two from:

- devices automatically detected and configured when first attached/plug and play
 - it is nearly impossible to wrongly connect a device
 - USB has become an industrial standard
 - supported by many operating systems
 - USB 3.0 allows full duplex data transfer
 - later versions are backwards compatible with earlier USB systems
 - allows power to be drawn to charge portable devices
- [2]**

(c)

Description of stage	Sequence number
the instruction is copied from the Memory Data Register (MDR) and placed in the Current Instruction Register (CIR)	3
the instruction is executed	6
the instruction is decoded	5
the address contained in the Program Counter (PC) is copied to the Memory Address Register (MAR)	1
the value in the Program Counter (PC) is incremented so that it points to the next instruction to be fetched	4
the instruction is copied from the memory location contained in the Memory Address Register (MAR) and is placed in the Memory Data Register (MDR)	2

[6]

90. M/J 15/P13/Q2**2**

Activity	First pass or second pass
any symbolic address is replaced by an absolute address	2
any directives are acted upon	1
any symbolic address is added to the symbolic address table	1
data items are converted into their binary equivalent	1
forward references are resolved	2

[5]

91. M/J 15/P13/Q5/a,b,c,d

5 (a) one mark for name of bus + one mark for description

address bus

- lines used to transfer address of memory or input/output location
- unidirectional bus

data bus

- used to transfer data between the processor and memory/input and output devices
- bidirectional bus

control bus

- used to transmit control signals
- e.g. read/write/fetch/ ...
- dedicated bus since all timing signals are generated according to control signal [6]

(b) (i) the program counter is incremented [1]

(ii) the data stored at the address held in MAR is copied into the MDR [1]

(iii) the contents of the Memory Data Register is copied into the Current Instruction Register [1]

(c) • the MAR is loaded with the operand of the instruction // loaded with 35
• the Accumulator is loaded with the contents of the address held in MAR
// the Accumulator is loaded with the contents of the address 35 [2]

(d) (i) • a signal
• from a device/program that it requires attention from the processor [2]

(ii) • at a point during the fetch-execute cycle ...
• check for interrupt
• if an interrupt flag is set/ bit set in interrupt register
• all contents of registers are saved
• PC loaded with address of interrupt service routine [4]

92. M/J 15/P13/Q7/a,b**7 (a) (i)**

Accumulator:

0	1	1	1	0	1	0	1
---	---	---	---	---	---	---	---

[1]

(ii)

Accumulator:

0	1	1	0	1	0	0	1
---	---	---	---	---	---	---	---

[1]

explanation

- content of 124 is 0 1 1 1 1 1 1 1
- this is equivalent to 127
- contents of 127 are 0 1 1 0 1 0 0 1

[2]

(iii)

Accumulator:

0	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---

[1]

explanation

- index register value = 6
- $120 + 6 = 126$
- contents of 126 placed in the accumulator

[2]

(b) 1 mark for each correct value in the table.

Accumulator	Memory address			
	320	321	322	323
	49	36	0	0
36				
37				
				37
49				
50				
			50	

[6]